Software Quality Assurance

Test Management

Test Estimation and Scheduling - Exercise

01 Quality Management

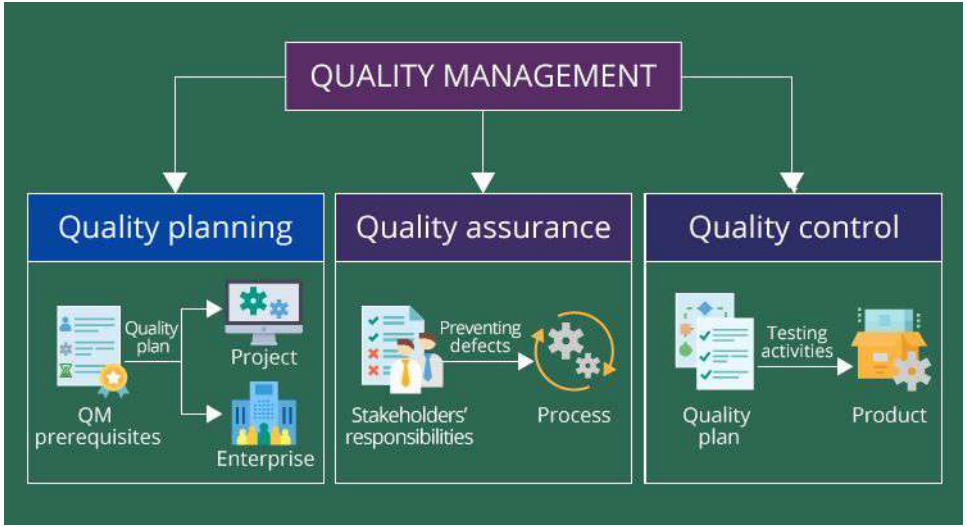
02 Quality Assurance

03 Quality Control

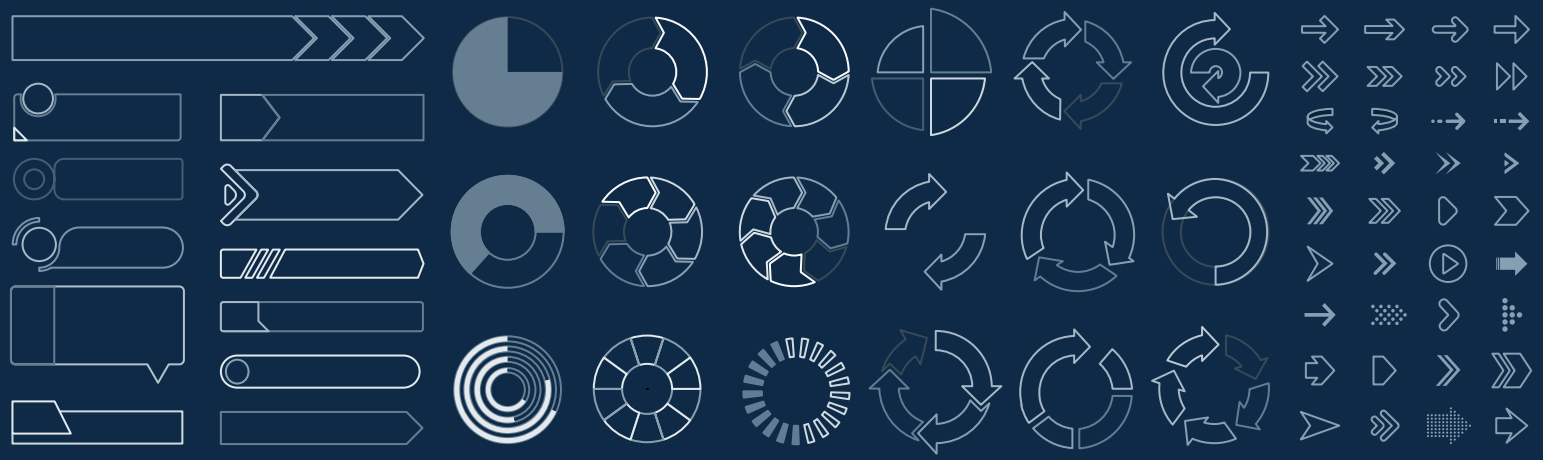
04 Team Management Case Study

Quality Management

Software Quality Management (SQM) is the management aspects of developing quality software



Quality Planning



Why a Test Plan ?

- Test Plan guides our thinking. It is like a rule book, which needs to be followed
- Test Plan helps people outside the test team such as developers, business managers, customers understand the details of testing
- Important aspects like test estimation, test scope, test strategy are documented in Test Plan, so it can be reviewed by Management Team and re-used for other projects.
- To help manage future changes. As the project evolves and situations change, we adapt our plans. Written test plans give us a baseline against which to measure such revisions and changes.

Purpose and Content of a Test Plan

A test plan outlines test activities for development and maintenance projects.

Planning is influenced by the;

- **Test Policy and the Test Strategy of the Organization**
- **Development Lifecycles and Methods being used (Sequential development models, Iterative and incremental development models, Agile model)**
- **Scope of Testing**
- **Objectives**
- **Risks**
- **Constraints**
- **Criticality**
- **Testability**
- **Availability of Resources**

Test planning activities in a test plan

- Determining the scope, objectives and risks of testing and then define the overall approach of testing
- Making decisions about what to test, the people and other resources required to perform the various test activities, and how test activities will be carried out
- Scheduling of test analysis, design, implementation, execution and evaluation activities, either on particular dates (e.g., in sequential development) or in the context of each iteration (e.g., in iterative development)
- Selecting metrics for test monitoring and control
- Budgeting for the test activities
- Test Tools

Test Estimation

Test Planning and Estimation

How to estimate?

There are a number of estimation techniques used to determine the effort required for adequate testing.

Two of the most commonly used techniques are:

- **The metrics-based technique**

Estimating the test effort based on metrics of former similar projects, or based on typical values

  - **Burndown Chart** [Effort is being captured and reported, and is then used to feed into the team's velocity to determine the amount of work the team can do in the next iteration]
  - **Defect Removal Models** [The volumes of defects and time to remove them are captured and reported, which then provides a basis for estimating future projects of a similar nature]

- **The expert-based technique**

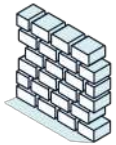
Estimating the test effort based on the experience of the owners of the testing tasks or by experts

  - **Planning Poker** [team members are estimating the effort to deliver a feature based on their experience]
  - **Wideband Delphi** [groups of experts provides estimates based on their experience]

Factors Influencing the Test Effort

Product Characteristics

- The risks associated with the product
- The size of the product
- The complexity of the product domain
- The requirements for quality characteristics (e.g., security, reliability)
- The required level of detail for test documentation
- Requirements for legal and regulatory compliance



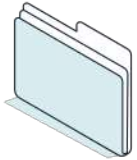
Development Process Characteristics

- The stability and maturity of the organization
- The development model in use
- The test approach
- The tools used
- The test process
- Time pressure

Factors Influencing the Test Effort

People Characteristics

- The skills and experience of the people involved, especially with similar projects and products (e.g., domain knowledge)
- Team cohesion and leadership

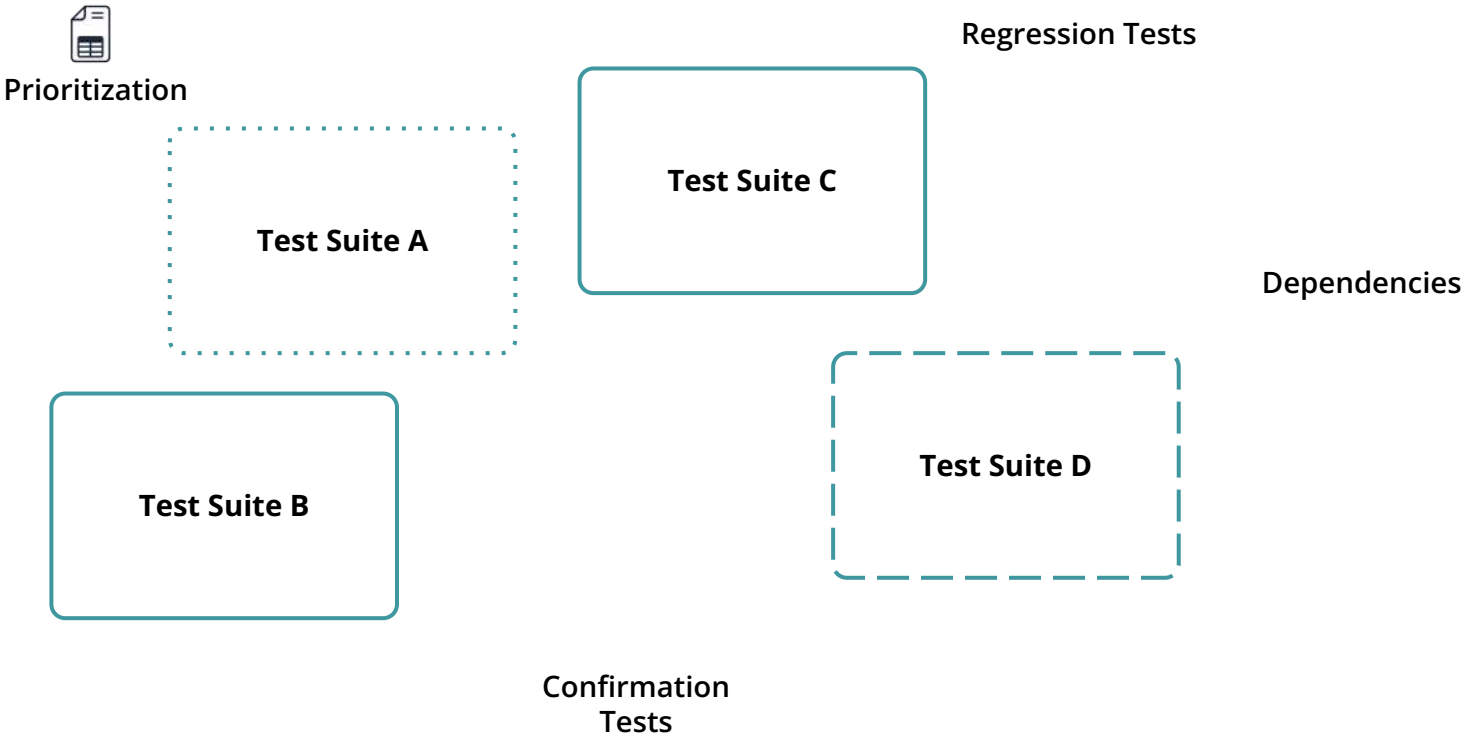


Test Results

- The number and severity of defects found
- The amount of rework required

Test Schedule

Test Execution Schedule



Test Execution Schedule

Test Suite A
Priority: Critical
Due Date: 19 Nov

Test Suite C
Priority: Low
Due Date: 22 Nov

Test Suite B
Priority: Medium
Due Date: 21 Nov

Test Suite D
Priority: High
Due Date: 20 Nov

Test Execution Schedule

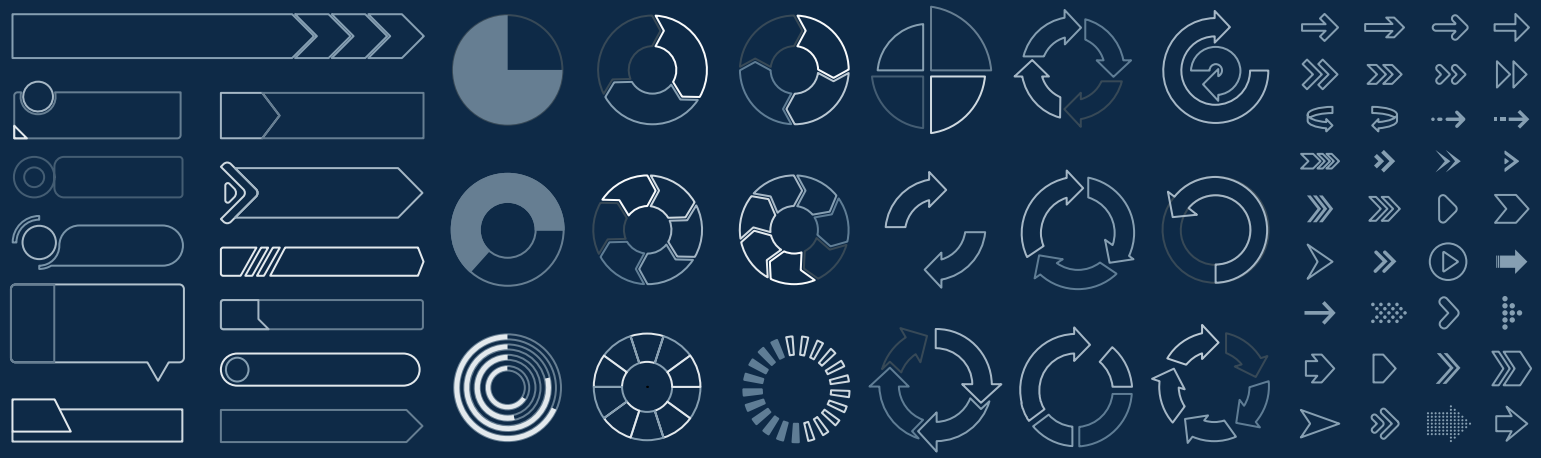
Test Suite A
Priority: Critical
Due Date: 19 Nov

Test Suite D
Priority: High
Due Date: 20 Nov

Test Suite B
Priority: Medium
Due Date: 21 Nov

Test Suite C
Priority: Low
Due Date: 22 Nov

Quality Assurance



Test Organization

Independent Testing

Levels of Independence

Degrees of independence in testing include the following (from low level of independence to high level)

No independent testers; the only form of testing available is developers testing their own code within the development team	Independent developers or testers within the development teams or the project team; this could be developers testing their colleagues' products	Independent test team or group within the organization, reporting to project management or executive management	Independent testers from the business organization or user community, or with specializations in specific test types	Independent testers external to the organization, either working on-site (in-house) or off-site (outsourcing)

Absence of independence → **Absolute independence**

Benefits and Drawbacks of Independent Testing

Benefits

- Likely to recognize different kinds of failures
- They can verify, challenge, or disprove assumptions made by stakeholders
- Independent testers of a vendor can report in an upright and objective manner about the system under test

Drawbacks

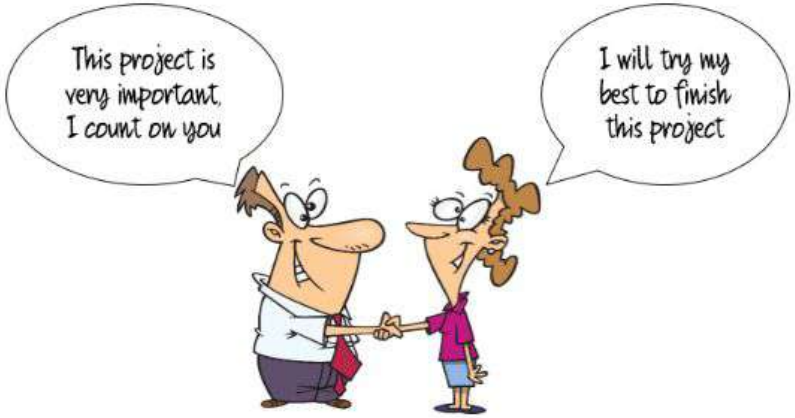
- Isolation from the development team
- project stakeholders might see the test team as a bottleneck and a source of delay;
- Developers may lose a sense of responsibility for quality.
- Independent testers may lack some important information

Role of a Test Manager and Tester

Test Manager - The person responsible for project management of testing activities, resources, and evaluation of a test object.

Tester - a person who performs testing

Test leader - On large projects, the person who reports to the test manager and is responsible for project management of a particular test level or a particular set of testing activities.



Tasks of a Test Manager and Tester

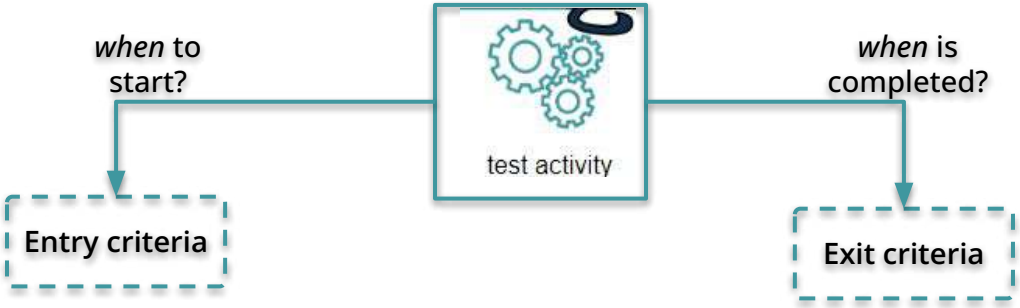
Typical Test Manager Tasks

- High level documents maintenance
- Estimate, Plan and Adapt planning
- Introduce metrics
- Make decisions
- Support testing process
- Coordinate
- Monitoring and test control
- Reporting to management and clients
- Initiate processes
- Budget questions
- Promote and advocate the testers, the test team, and the test profession within the organization
- Develop the skills and careers of testers

Typical Tester Tasks

- Reviewing and contributing to test plans from the tester perspective;
- Analyzing, reviewing and assessing requirements, user stories, specifications
- Identifying and documenting test conditions and test cases, capturing traceability between test cases, test conditions and the test basis
- Designing, setting up and verifying test environments(s)
- Designing and implementing test cases and test procedures, including automated tests
- Acquiring and preparing test data
- Executing the tests, evaluating the results and documenting deviations from expected results as defect reports;
- Automating tests as needed (for technical test specialists)
- Reviewing tests developed by others

Entry Criteria and Exit Criteria

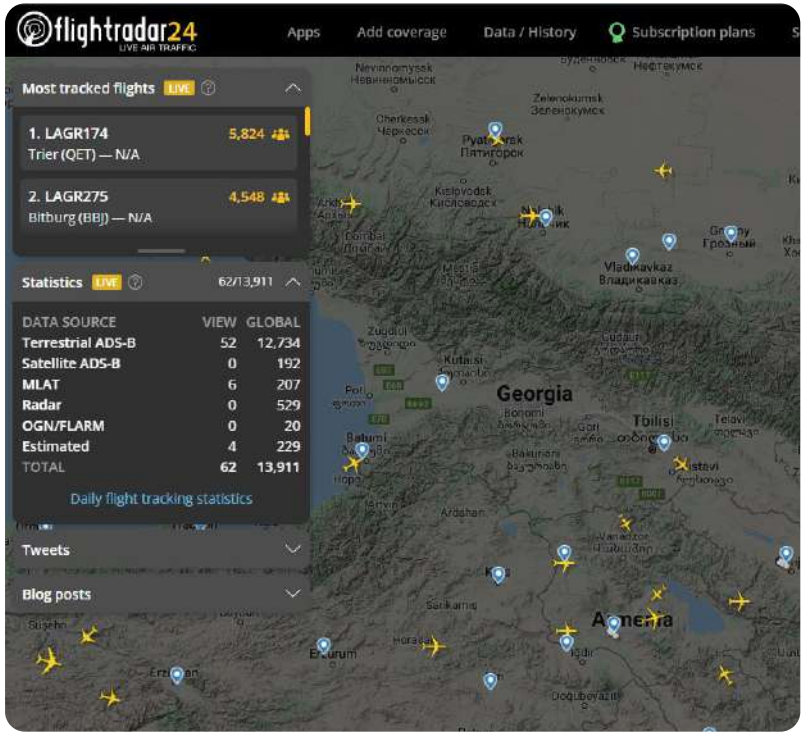


- Availability of testable requirements, user stories, and/or models (e.g., when following a model-based testing strategy)
- Availability of test items that have met the exit criteria for any prior test levels
- Availability of test environment
- Availability of necessary test tools
- Availability of test data and other necessary resources

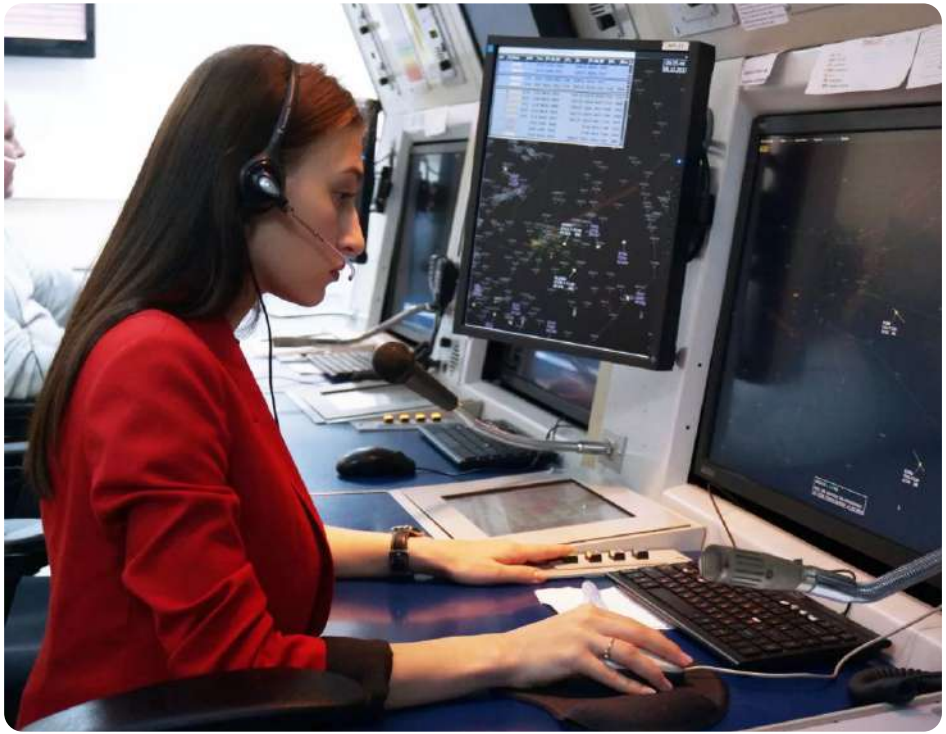
- Planned tests have been executed
- A defined level of coverage (e.g., of requirements, user stories, acceptance criteria, risks, code) has been achieved
- The number of unresolved defects is within an agreed limit
- The number of estimated remaining defects is sufficiently low
- The evaluated levels of reliability, performance efficiency, usability, security and other relevant quality characteristics are sufficient

Test Monitoring and Control

Test Monitoring and Control



Monitoring



Control

Metrics Used in Testing

Common test metrics include:

- Percentage of planned work done in test case preparation (or percentage of planned test cases implemented)
- Percentage of planned work done in test environment preparation
- Test case execution (e.g., number of test cases run/not run, test cases passed/failed, and/or test conditions passed/failed)
- Defect information (e.g., defect density, defects found and fixed, failure rate, and confirmation test results)
- Test coverage of requirements, user stories, acceptance criteria, risks, or code
- Task completion, resource allocation and usage, and effort
- Cost of testing, including the cost compared to the benefit of finding the next defect or the cost compared to the benefit of running the next test

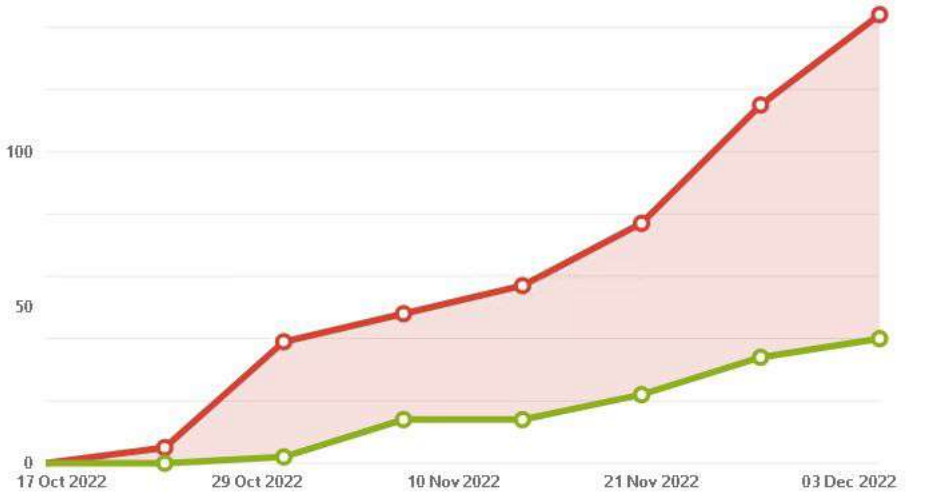
	Status for Release 10.0.11.45.0.3							
	Test Execution							
s/Blockers	Total	Executed	Passed	Failed	Blocked	Execution rate	Number of Covered Test Cases	Test To
	3180	3143	2192	742	37	99%	100%	
	Manual = 1076 (33.84%) Automated = 2104 (66.16%)		68.93%	23.33%	1.16%			
	945	908	847	61	37	96%	100%	
	57	53	47	6	4	93%	100%	
ation is not fully	57	53	47	6	4	93%	100%	Manual
	750	717	676	41	33	96%	100%	

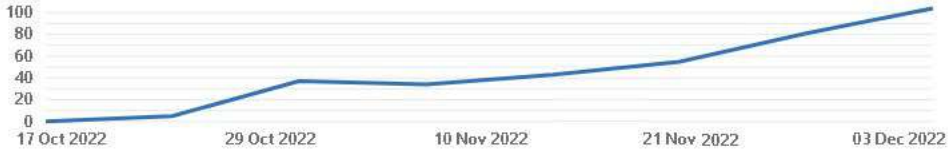
Two Dimensional Filter Statistics: ALL Issues

Status	Critical	High	Low	Medium	None	T
UNDER INVESTIGATION	0	2	6	12	0	20
CONFIRMED	1	41	17	26	0	85
RESOLVED	0	4	3	8	1	16
CLOSED	2	14	2	6	0	24
Total Unique Issues:	3	61	28	52	1	145

Grouped by: Severity

Showing 4 of 4 statistics.





Issues in the last 45 days (grouped weekly) View in Issue Navigator

- Created issues (144)
- Resolved issues (40)

Purposes, Contents and Audiences for Test Reports

TEST PROGRESS REPORT

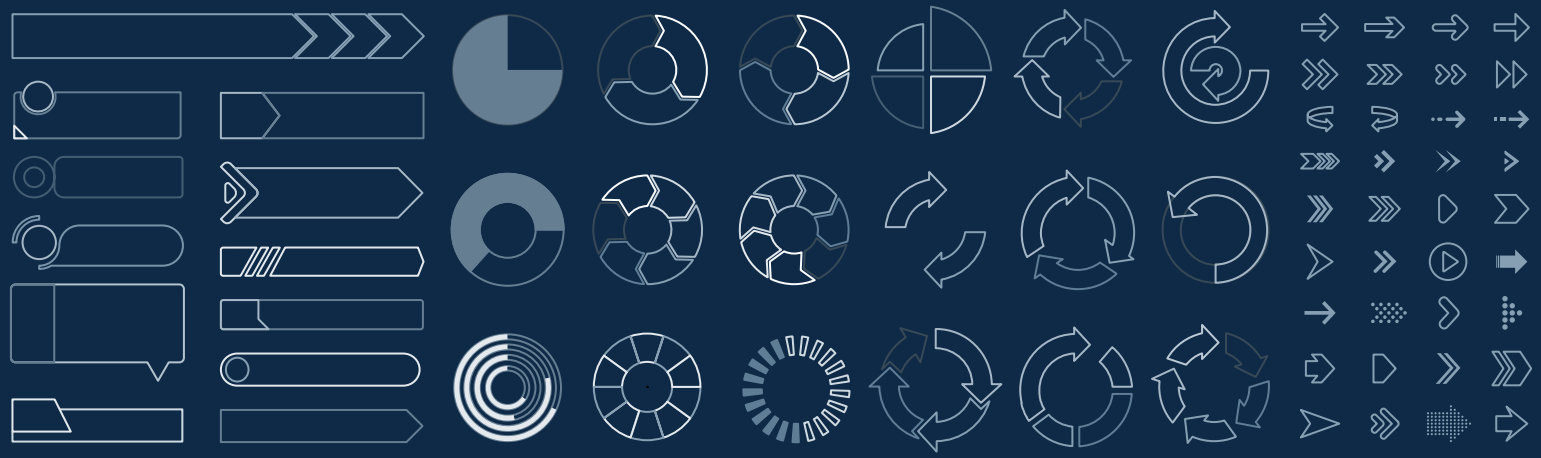
- The status of the test activities and progress against the test plan
- Factors impeding progress
- Testing planned for the next reporting period
- The quality of the test object

TEST SUMMARY REPORT

- Summary of testing performed
- Information on what occurred during the test period
- Deviations from the plan
- The status of testing and product quality
- Blocking Factors
- Metrics of defects, test coverage, activity progress, and resource consumption
- Residual risks
- Reusable test work products produced

GREEN

Quality Control



Verification and Validation

Verification and Validation

Verification and Validation is the process of investigating that a software system satisfies specifications and standards and it fulfills the required purpose

Verification **:** Are we building the product right?

Validation : Are we building the right product?

Verification

Verification is the process of checking that a software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements that we have.

Verification is Static Testing.

Activities involved in verification:

- Inspections
- Reviews
- Walkthroughs
- Desk-checking

Validation

Validation is the process of checking whether the software product is up to the mark or in other words product has high level requirements. It is the process of checking the validation of product i.e. it checks what we are developing is the right product. it is validation of actual and expected product.

Validation is the Dynamic Testing.

Activities involved in validation :

- Black box testing
- White box testing
- Unit testing
- Integration testing

Risk and Testing

Definition of Risk

Risk - a factor that could result in future negative consequences.

Product and Project Risks

PRODUCT

- Software might not perform its intended functions according to the specification
- Software might not perform its intended functions according to user, customer, and/or stakeholder needs
- A system architecture may not adequately support some non-functional requirements.
- A particular computation may be performed incorrectly in some circumstances.
- A loop control structure may be coded incorrectly.
- Response-times may be inadequate for a high-performance transaction processing system.
- User experience feedback might not meet product expectations.

PROJECT

- Project issues
- Organizational issue
- Political issues
- Technical issues
- Supplier issues

Risk-based Testing and Product Quality

Analyze what can go wrong	Determine the priority of risks
Implement actions to mitigate those risks	Make contingency plans

To determine

- the test techniques to be used.
- the particular levels and types of testing to be performed.
- the extent of testing to be carried out for the different levels and types of testing.
- whether any activities in addition to testing could be employed to reduce risk.

To prioritize testing

Defect Management

Defect Management

?

Does not work!

REJECTED

DEFECT REPORT

ID
Date
Title
Summary
Defect Priority
Defect Severity
State of the defect report
Expected and actual result
Environment and configuration
The development lifecycle phase
A defect description to enable reproduction, including logs, database dumps, screenshots, or recordings

FIXED

Summary: [Gmail] A counter of total pages is displayed an incorrect number if user opens a file from attachments

Description:

Preconditions:

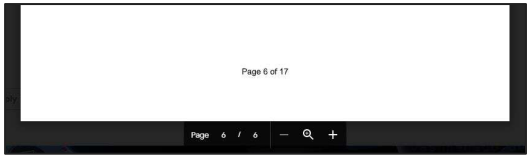
1. Please download the .docx file attached and do not change
2. Open a browser window (a browser should be updated to the latest version)

Steps:

1. Open Gmail window
2. Compose a new email
3. Attach the downloaded file
4. Send the email
5. Open the email, find the file in the section for attachments and click on it to view its content
6. Check the total pages counter

Expected Result: total pages counter displays the actual number 6

Actual Result: total pages counter displays the incorrect number 17



Please note that this issues has been reproduced in other browsers, e.g. Opera, Firefox

Workaround: manually update the total pages counter in Word or Google Docs and save the file

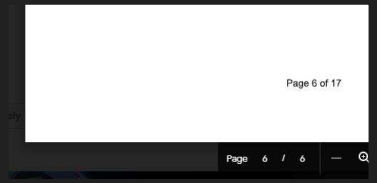
Environment: Windows OS, Google Chrome: Version 100.0.4896.88 (Official Build) (64-bit)

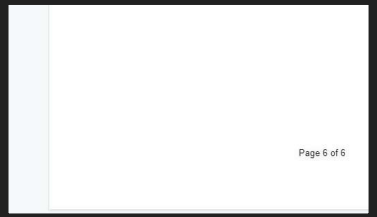
Issue type: Bug

Severity: Low

Labels: Gmail, DOCX

Attachments:





Jira Software | Your work | Projects | Filters | Dashboards | People | Apps | Create | Search

PLANNING
- Roadmap
- Board

DEVELOPMENT
- Code
- Releases

OPERATIONS
- Deployments
- On-call

Project pages

You're in a team-managed project

Projects / [illegible] / [illegible] / [illegible]

[illegible] Reconciliation messages from IoxE's traffic files is too long

Attach | Add a child issue | Link issue

Description

[illegible]

Environment

None

Attachments (1)

Activity

Show: All | Comments | History | Newest first

Add a comment...

Pro tip: press M to comment

6

RELEASED

Details

SOLUTION/CORE	None
Assignee	[illegible]
Due date	None
Priority	High
Labels	None
Development	Create Branch
Reporter	[illegible]
Automation	Rule executions

Created 8 days ago
Updated 3 days ago

Configure

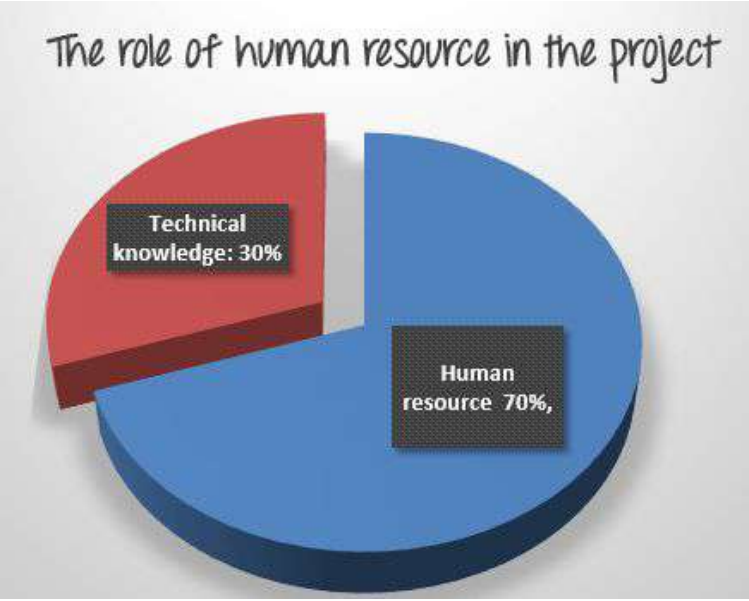
Team Management

What are the People skills

Collaboration

Presentation

Conflict-Resolution



People skills Helps you

Master P

1
Effective Communication
2
Building Relationships
3
Influencing People
4
Motivating People
5
Handling Ethical Issues

Exercise

Team Management Case Study

Case 1

Two senior team members have a disagreement on project approach on operational level that is affecting the team's morale and productivity, What will be your approach to address this matter?

Case 2

The project scope has changed due to unforeseen circumstances Whatwillbe the challenges you will face and what youdo as a QA manager? A team members are showing signs of burnout. What will you do as a QA team lead?

Case 3

The team has successfully completed a challenging project. They have put lots of additional effort, late hours to deliver the project with expected quality. What will be your immediate steps and approach to keep the team moral as it is?

Case 4

Your organization is decided to implement set of new process enhancement (Such as 5S or CMMi or ISO standards) to improve efficiency and quality of each department/projects. There is significant resistance from the team. What will be your responsibilities as a manager

Solutions

Two senior team members have a disagreement on project approach on operational level that is affecting the team's morale and productivity, What will be your approach to address this matter?

Key Points:

Team conflict is natural. But by practicing the conflict management skills we outline here, you'll be able to spot and deal with issues between team members before they escalate.

To avoid team conflict:

- Be proactive.
- Observe.
- Be impartial.
- Step in when needed.
- Avoid assumptions.
- Be patient.

If team conflict persists:

- Speak to team members individually.
- Bring people together.
- Ask the wider team for ideas.
- Draw up a plan.
- Follow up.

Solutions

The project scope has changed due to unforeseen circumstances Whatwillbe the challenges you will face and what youdo as a QA manager? A team members are showing signs of burnout. What will you do as a QA team lead?

Key Points:

Burnout is a state of physical and emotional exhaustion that occurs from long-term stress at work. It's most common when you have worked in a physically or emotionally demanding role for a sustained period.

To avoid team burnout:

- Reset your priorities and stick to them
- Set boundaries on communication
- Step back, get context, and adapt your approach
- Find your source of burnout and take back control
- Ask for help and support from others

Help your team deal with burnout:

- Learn about burnout
- Clearly define roles & responsibilities
- Practice workload management
- More work time, less meetings
- Become a servant leader

Solutions

The team has successfully completed a challenging project. They have put lots of additional effort, late hours to deliver the project with expected quality. What will be your immediate steps and approach to keep the team moral as it is?

Key Points:

First, focus on your own morale. Then identify why team morale is low, and choose appropriate strategies for rebuilding it.

To keep team morale:

- Focus on Little Things
- Celebrate each achievement
- Step Back and focus on big picture
- Stay Strong and engage
- Get continuous feedback

Help your team to retain team morale:

- Reconnecting with your team
- Build Development plan for your team
- Improving the workplace
- Rebuilding confidence
- Look for rewards the team

Solutions

Your organization is decided to implement set of new process enhancement (Such as 5S or CMMi or ISO standards) to improve efficiency and quality of each department/projects. There is significant resistance from the team. What will be your responsibilities as a manager

Key Points:

First, focus on your own morale. Then identify why team morale is low, and choose appropriate strategies for rebuilding it.

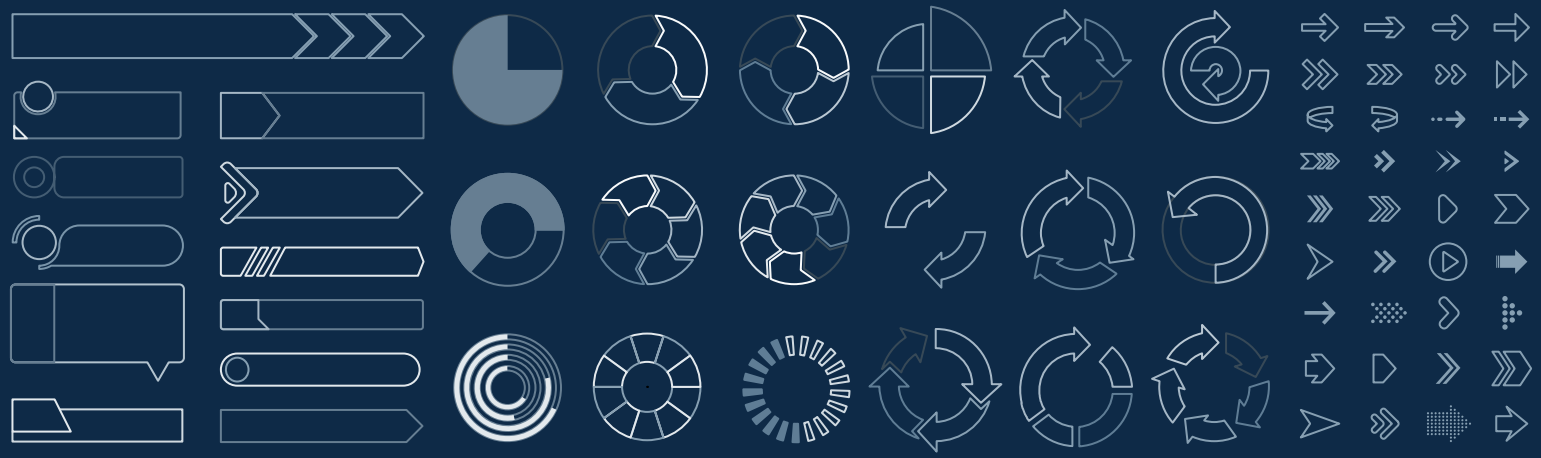
reasons for resistance to change:

- Lack of trust
- Poor communication
- Emotional response
- Fear of failure
- Constant change

Help your team to minimize resistance to change:

- Communicate early and often
- Listen to employees
- Educate employees on the value of the change
- Name emotions
- Provide ongoing support

Quality Assurance Plan Case Study



Create your Own Test Plan

- Introduction
- Requirement for Testing
- Product Risks
- Test Strategy
- Testing Types
- Test Methodology for Test Types
- Entry /Exit Criteria for Test Types
- Test Schedule
- Team Structure
- Roles and Responsibilities
- Results Reporting
- Quality Gates

https://docs.google.com/document/d/1fvs3g33Wsx_1Ln1SHW5ECkkjrp6QZFueM5Fo38B5gvI/edit?usp=sharing

Test Management in Agile

Test Estimation and Scheduling - Exercise

01 Introduction to Agile

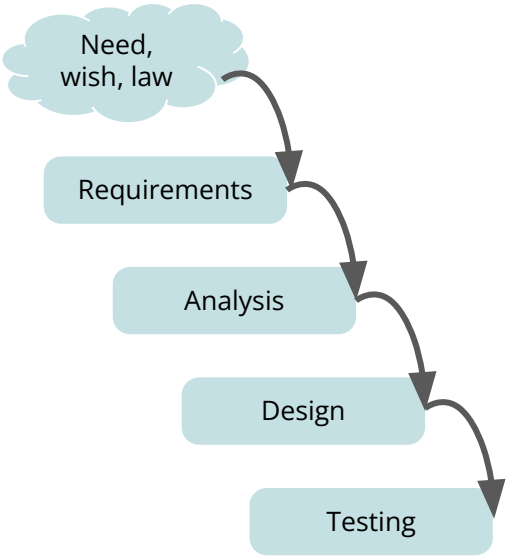
02 Agile Development in Practise

03 Quality Management in Agile

04 Case Study

Waterfall Development

The waterfall model is a software development methodology strategy which relies on following a set sequence of actions from top to bottom in the form of a waterfall. Its stages are closely aligned with the software development life cycle (SDLC) from which it evolved. (Source:verasseti.com)



The waterfall model has a natural timeline where tasks are executed in a sequential fashion.

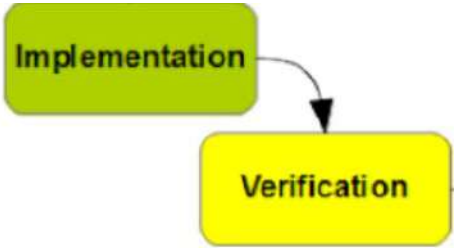
\+ It is easy to manage and the process is well-documented.

\- It is hard to get a feedback passed up the waterfall. Test activities only occur after all other development activities have been completed. **High cost of defects**.

Waterfall Development (contd.)

Linear Sequential flow where you complete one phase

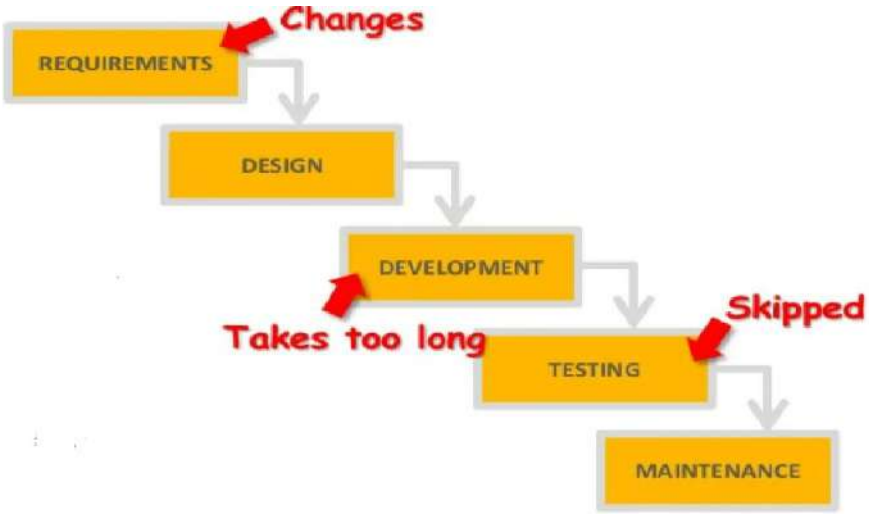
before moving to next phase.



Rarely aim to revisit a ‘phase’ once it is completed.

“Better do it right the first time”

Waterfall Development (contd.)



However …

You will not see a value until end of the the project

Stakeholders will not see the application until end of the project

Requirement changes are not welcomed

No testing until end of the development

What if you found a critical defect ??????

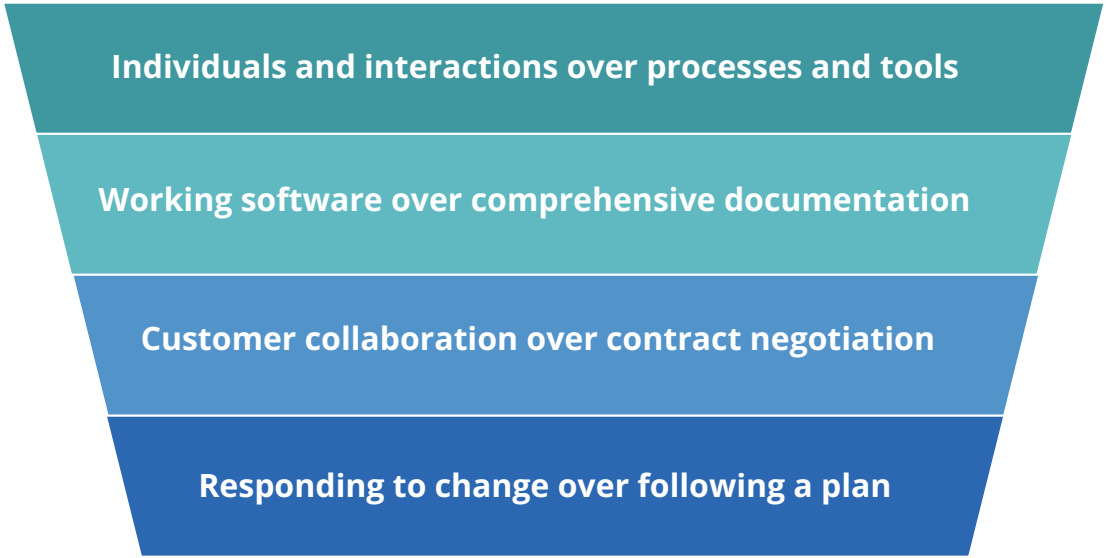
What if your deadlines cannot be achieved??????

Issues with Waterfall

- QA don’t get a live system to carry out their test until the later stage of the release.
- If an issue found out in the late stages, the delivery dates may be pushed back to resolve them.
- Time and effort waste due to such issues were high.
- QA team would get a high workload in the final stages of the release.
- Not much interaction were done between the BA/Dev & QA.

Agile development

The Manifesto for Agile Software Development



In 2001, these seventeen software developers met at a resort in Snowbird, Utah to discuss these lightweight development methods. Together they published the **Manifesto for Agile Software Development**.

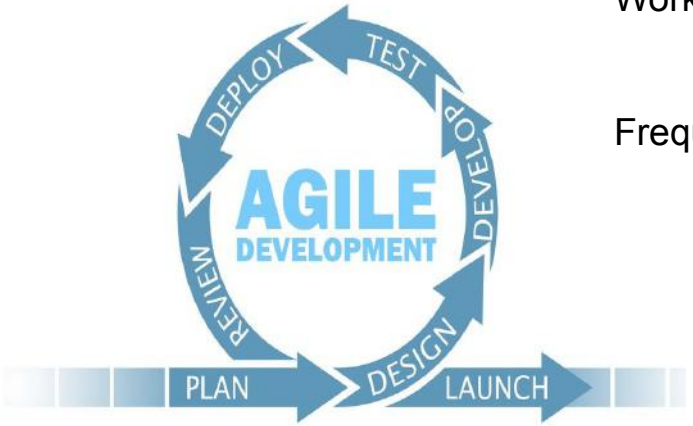
What is Agile?

Project management and software development approach

Iterative development

Incremental , Frequent Delivery

Quality Driven



Working software primary focus

Frequent customer feedback

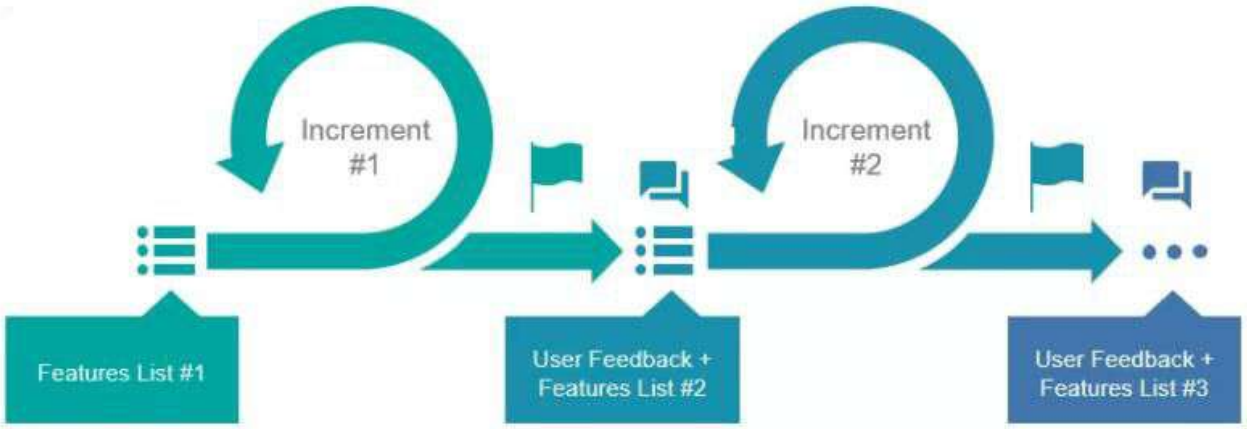
Cross functional and self -organizing teams

Collaboration

Not a Process, it's a ***philosophy*** , a ***set of values***

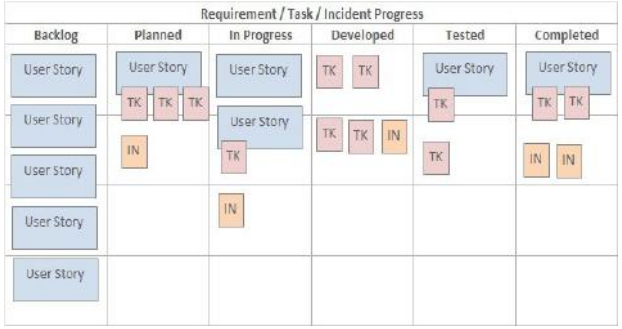
Agile Process

- Development team implement set of feature list
- User Feedback is received
- Development path updated based on the user feedback
- Given feedback incorporated into next delivery

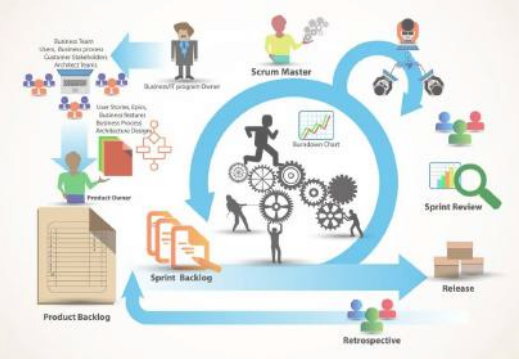


Key Agile Methodologies

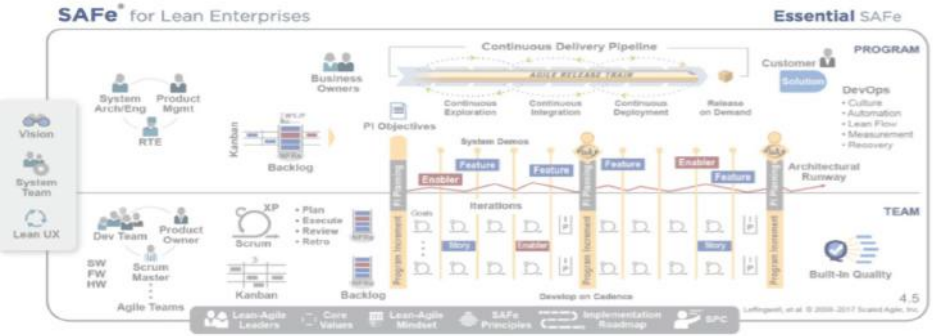
Kanban



Scrum

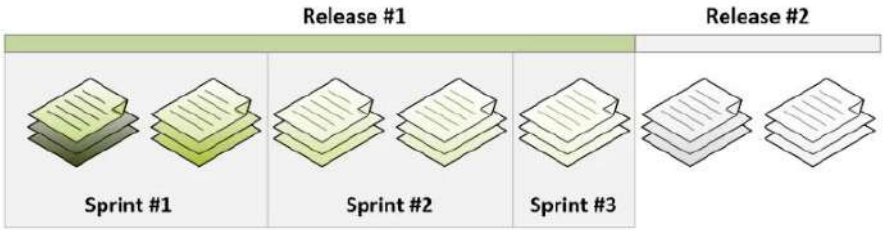


SAFe



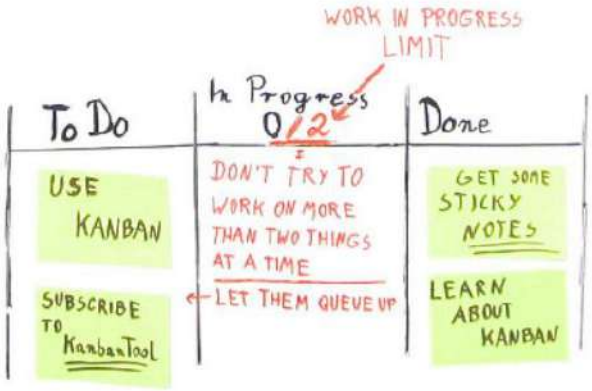
Scrum

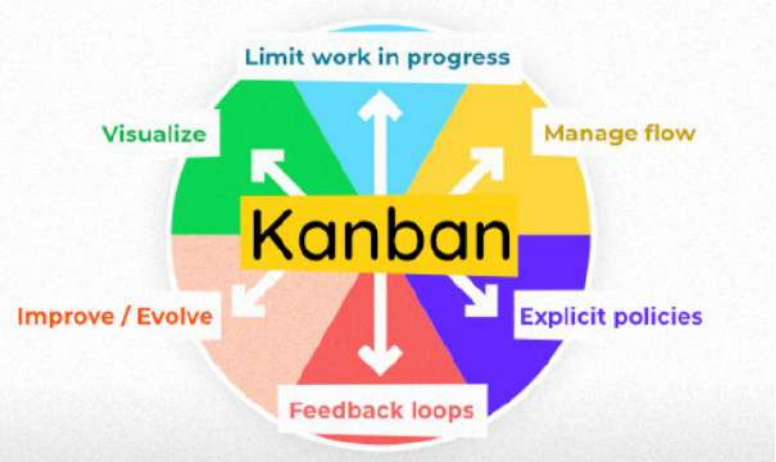
- A lightweight agile process tool
- Split organization in to small , cross functional teams
- Self – organizing teams
- Split release in to fixed 2 weeks sprints
- Split work in to small ,concrete deliverables based on priority and effort



Kanban

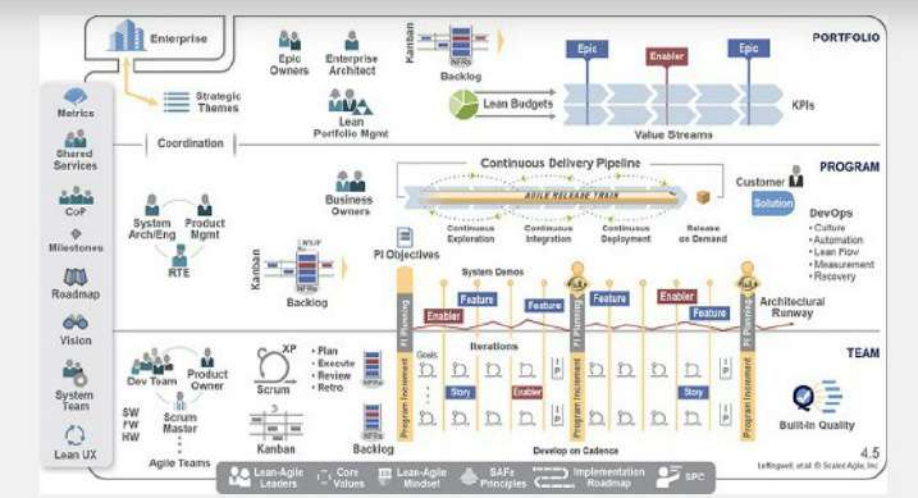
- Popular framework used to implement agile and DevOps software development.
- Is all about visualizing your work, limiting work in progress, and maximizing efficiency (or flow).
- Work items are represented visually on a kanban board.
- Allowing team members to see the state of every piece of work at any time.
- Requires real-time communication of capacity and full transparency of work.
- Kanban teams focus on reducing the time a project takes (or user story) from start to finish.
- Continuous flow.





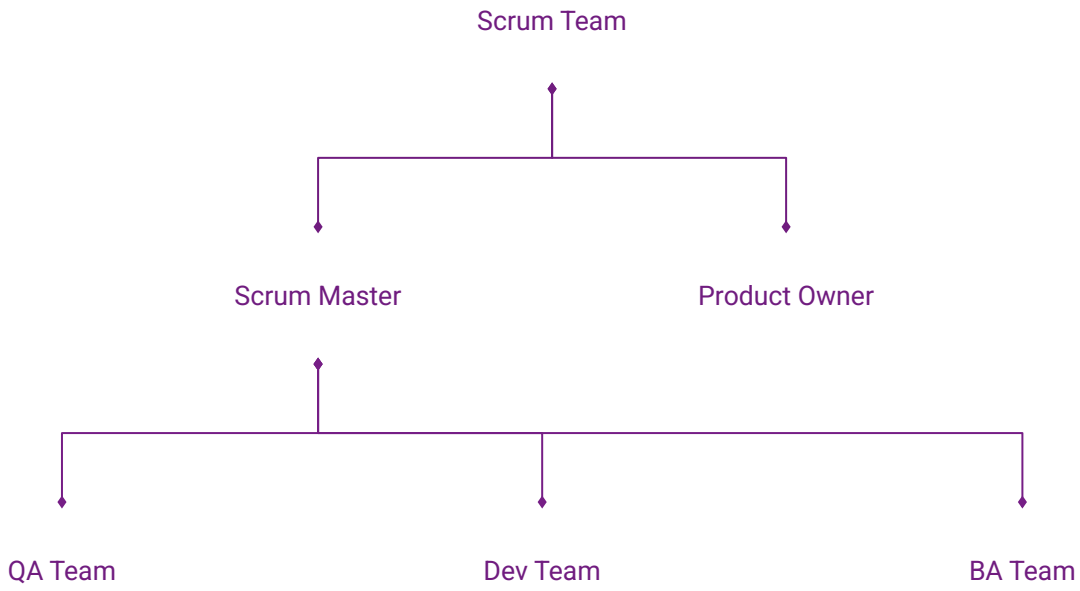
SAFe

- Promotes alignment, collaboration, and delivery across large numbers of agile teams.
- Project management in an organization using SAFe happens at three levels.
- At the individual team level, it's basically Scrum business as usual.
- Small teams have specific goals and areas of responsibility.
- They release iterations after each Sprint.
- The only significant change is that these small teams now roll up into programs.



Agile Team Structure

- Team size is about 7-10



Product Owner

Product Owner - Responsibilities

- Work with customers, users, and organizational stakeholders to understand their needs, and then translate that into the product backlog
- Determining which features and functionalities will develop, and the order
- Ensures that the team has a clear vision of sprint goals and overall objectives
- Sole authority to decide what will develop in Sprint
- Responsible for the overall success of the solution that the scrum team develops
- Manage the economics and ensure that he is getting sufficient Return of Investment from the work

Product Owner - Skills

Domain Skills

The product owner is responsible for working with stakeholders to create a product vision and roadmap. He is also responsible for creating product backlog and acceptance criteria for stories. In order to do all this work, the product owner needs to be very strong in Domain.

People Skills

The product owner works with different stakeholders to create a common vision for the product. Different stakeholders may have different and conflicting needs. The product owner needs to have good people skills where he could negotiate and build consensus.

Decision Making

The product owner must be empowered to make his own decisions. He should be able to make hard decisions when needed. There are often situations where the product owner needs to make a trade-off between scope, time, and budget.

Accountability

The product owner is ultimately accountable for the product that gets created, and a significant part of the success or failure of product rest on his shoulders. As such, the product owner should take the accountability of the product. He should be commit and available to the Scrum Team, and to other stakeholders.

Scrum Master

FOSTERING COMMUNICATION
Creating channels and translating lingo

PROTECTING THE TEAM
Act as a buffer for disruptive PMs and Management

TOOL MAINTENANCE
Jira administration

REPORTING
Creating burndown charts, velocity, etc.

MEETING FACILITATION
Standups, retros, sprint planning, etc

AGILE COACHING
1 on 1 and at the organizational level

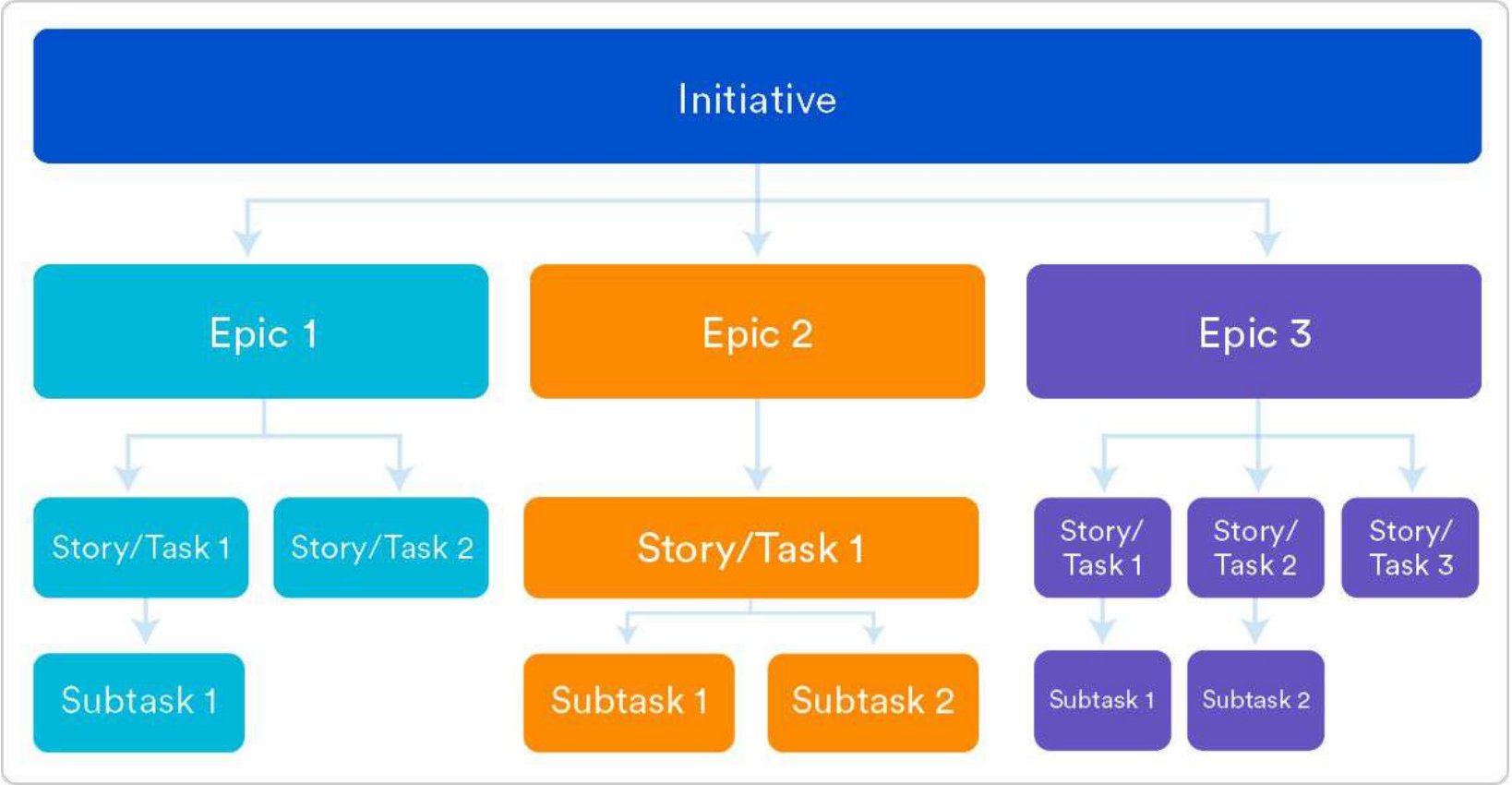
TEAM SUPPORT
From settling disagreements to coffee runs

REMOVE BLOCKERS
Superhero time

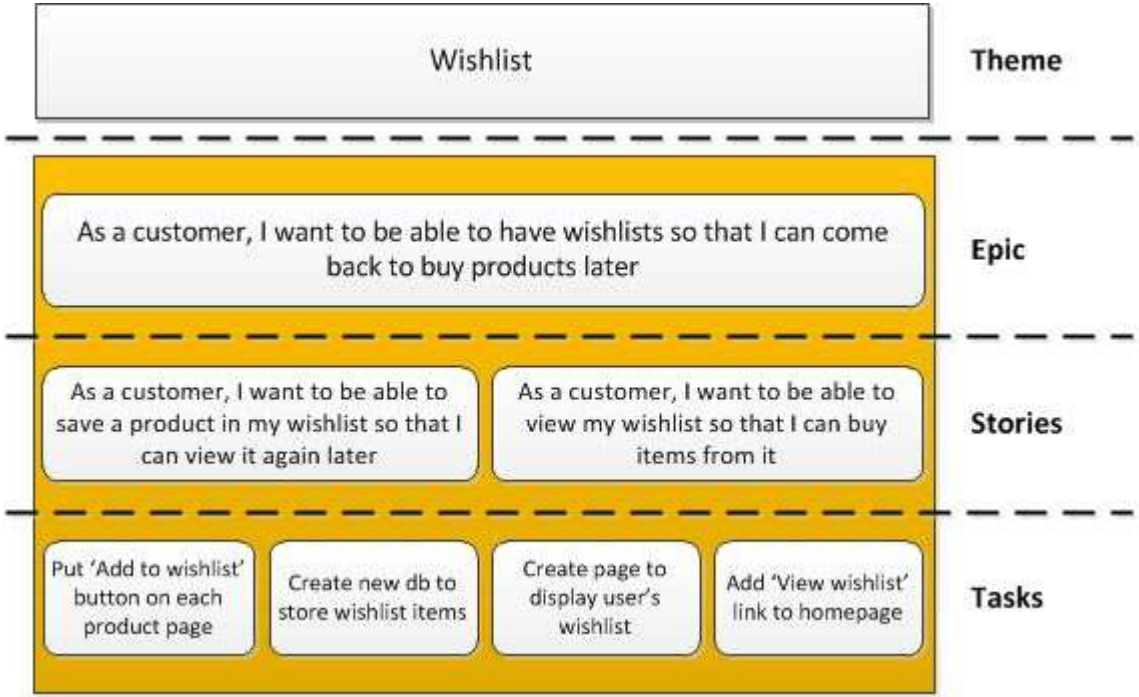
Scrum Master - Responsibilities

- Ensure that everyone on the Scrum Team understands sprint Goals, Scope, and product increments.
- Makes sure that the team is following the scrum process
- Primary job of a Scrum Master is to ensure the removal of any impediments that may impact the dev team’s progress
- The SM facilitates Scrum events as requested or needed
- Scrum Master coaches the team to be committed to the Sprint goal, and not to take any requirements from anyone other than the Product Owner.

Epics/ User Stories/ Tasks



Epics/ User Stories/ Tasks



Agile Ceremonies and Process

- Sprint is around 2- 3 weeks

Backlog Grooming

PO and the Team analyze the backlog and prioritize the backlog

Daily Standup

Team have a quick daily standup to discuss the progress and blocks. Inform what was done previous day and what is planned for today.

Sprint Start

Sprint End

Week before sprint start

Daily

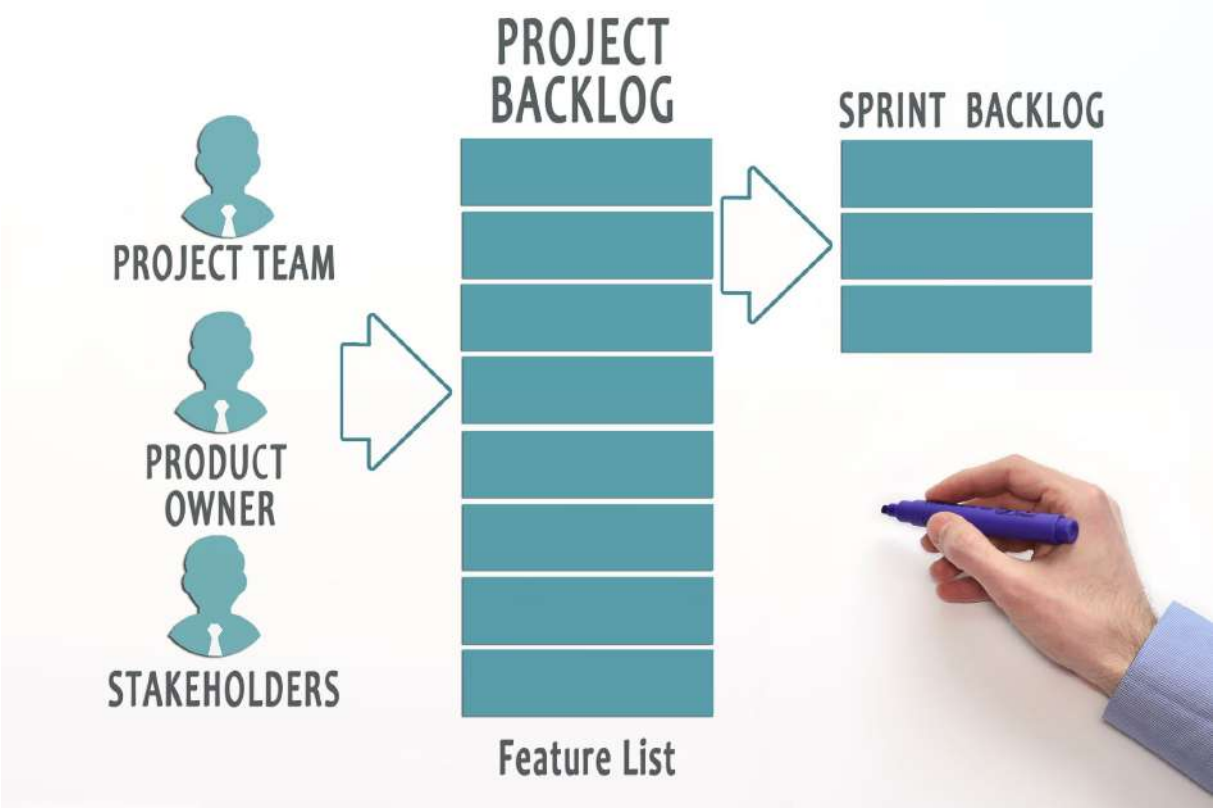
Sprint Planning

Assign the tasks to each team member and allocate relevant estimated times

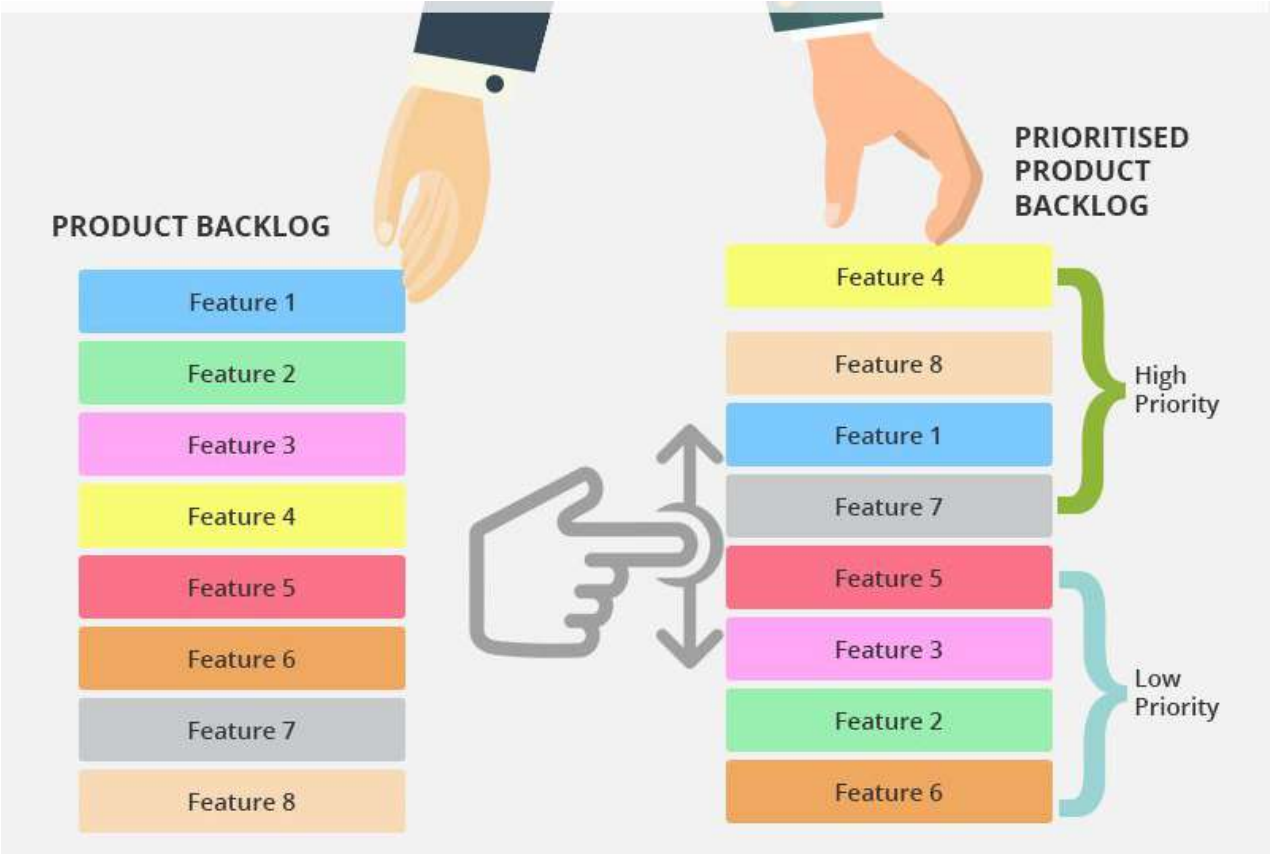
Retrospective

Analyze all the allocated tasks are completed and discuss the success of the sprint. Perform a client demo.

Backlog Grooming



Backlog Grooming



Agile Ceremonies and Process

- Sprint is around 2- 3 weeks

Backlog Grooming

PO and the Team analyze the backlog and prioritize the backlog

Daily Standup

Team have a quick daily standup to discuss the progress and blocks. Inform what was done previous day and what is planned for today.

Sprint Start

Sprint End

Week before sprint start

Daily

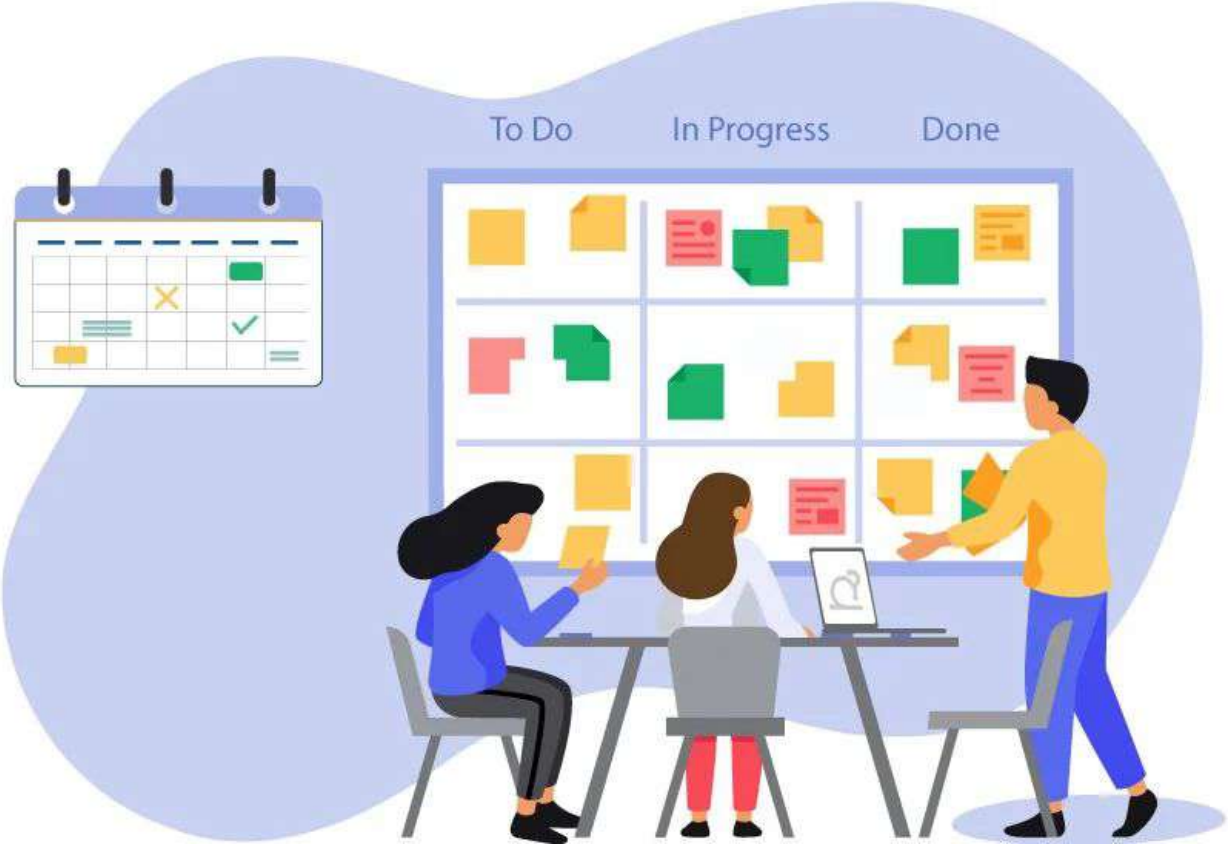
Sprint Planning

Assign the tasks to each team member and allocate relevant estimated times

Retrospective

Analyze all the allocated tasks are completed and discuss the success of the sprint. Perform a client demo.

Sprint Planning



Story Points

A story point is a theoretical measure that shows the difficulty level of any work on a project.

© 2023, Lemberg Solutions LLC. All rights reserved

Story points estimation

0 - takes minutes to implement or already completed

Item is already completed or is so small that it doesn't make sense to even give it a size number.

Example: verify whether discussed fix had indeed been applied.

0.5 - easy, no risk/complexity fix or minor feature

Should not take longer than an hour to implement and verify.

Example: implement confirmation dialog; remove a UI element (a button or a field).

1 - easy and minimal complexity; takes few hours of efforts to implement

A straightforward task, could be carried on by most inexperienced members of the team. Minor efforts to implement and verify.

Example: add a field(s) to UI/API request/DB; forgot password form/request.

2 - still a relatively straightforward, while more complex and higher in volume than "1"

Could potentially take 1-2 days to implement and verify.

Example: user sign in functionality; navigation; welcome screen.

3 - moderate complexity or size

An average piece of work, that takes some effort but does not introduces risks or technical debt.

Example: user sign up functionality; view user profile.

5 - moderate to high complexity/size, some risks

Could take a week to implement and verify, a more complex and higher in volume task that requires some analysis and thorough implementation.

Example: creating custom UI control (i.e. graph, timeline, etc); advanced search; object edit history.

8 - high complexity/size, moderate risks/uncertainty

This is a maximum size of a story that is acceptable to include in the sprint, should be broken down into smaller stories if possible.

Example: refactor data storage layer for a big project; implement map with various data and filters on it.

13 — high risk/complexity/volume, high risks/uncertainty

Should not be included in the sprint, broken into smaller and more detailed stories instead.

Example: implement paginated news feed that supports audio or video previews and various social activity and interaction; add tablet support for a phone app.

20, 40 — very high risk/complexity/volume, too many uncertainties to estimate properly

Should not be included in the sprint, broken into smaller and more detailed stories instead.

Example: implement paginated news feed that supports audio or video previews and various social activity and interaction; add tablet support for a phone app.

100, ∞ - Indicates that a story is so large, vague and ambiguous that it's pointless to estimate until further details are given.

Planning Poker

T-Shirt Sizing

Estimation Example

Epic/Feature	Sub - Task	Estimate, SP
User Authorization 11.5 SP	· Sign In · Sign In w/ FB · Sign In w/ Google · Sign Up · Sign Out · Forgot Password	1 SP 3 SP 2 SP 2 SP 3 SP 0.5 SP
User Profile 6 SP	· View User Profile · Edit User Profile (Change Photo/Name/Email/Phone) · API Requests	2 SP 3 SP 1 SP
Messaging 14 SP	· View Message Inbox · View Message Details · Send Message · View Sent Messages · API Requests/Lib Integration	3 SP 2 SP 3 SP 1 SP 5 SP
Maps 18.5 SP	· Google Maps Integration · API Requests (pulling employer data) · Draw Custom Pins · Zoom In/Out Depending on Data (clustering) · Maps Search · My Location	5 SP 3 SP 2 SP 3 SP 5 SP 0.5 SP
Employer Profile 10 SP	· Employer Data · List of Jobs · Job Details · Apply for Offer · View on Map · Pagination	1 SP 1 SP 2 SP 2 SP 2 SP 2 SP
	Total estimate	60 SP

Your Workspace > Your Board

Sprint November

30 December 15:17 – 30 December 15:17 • 33% • 4 /12 Task closed • 1/4 Completed PBIs • 30/58 Completed Hours

+ Task + Status

Bulk Actions | Move active PBIs | Export | Search for Tasks or PBIs

USER HISTORY (5)

#P12 Implement the notifications to users
IN PROGRESS
30 December 15:17 7h/18h
Improvements

#P7 Integration with customer customer records
IN PROGRESS
30 December 15:17 17h/25h
Product

#P6 Optimize the speed of page loading speed
IN PROGRESS
30 December 15:17 17h/26h

TO DO (4)

#T47 Redesign the template for responsive
0/4h
27 December
Improvements

+ NEW TASK

#T45 Tests
2/8h
10 November
Product

+ NEW TASK

#T55 Optimize images
0/3h
30 December
Web

#T56 Check server parameters
0/5h
28 December
Web

+ NEW TASK

DOING (5)

#T46 Define a notification template notification
1/8h
26 December
Improvements

+ NEW TASK

#T47 Development
7/8h
10 November
Product

+ NEW TASK

#T58 Optimize the configuration file for the cache
3/4h
23 December
Web

+ NEW TASK

VERIFYING (1)

+ NEW TASK

#T47 Design
7/8h
10 November
Product

+ NEW TASK

+ NEW TASK

DONE (4)

#T52 Search tools for sending notifications to users
6/6h
27 December
Improvements

+ NEW TASK

#T57 Test database of customers to which marketing notifications are sent.
1/8h
27 December
Users

+ NEW TASK

#T53 Compare with other wordpress themes 2/2h
30 December
Product

#T54 Delete plugins
1/1h
30 December
Product

+ NEW TASK

Agile Ceremonies and Process

- Sprint is around 2- 3 weeks

Backlog Grooming

PO and the Team analyze the backlog and prioritize the backlog

Daily Standup

Team have a quick daily standup to discuss the progress and blocks. Inform what was done previous day and what is planned for today.

Sprint Start

Sprint End

Week before sprint start

Daily

Sprint Planning

Assign the tasks to each team member and allocate relevant estimated times

Retrospective

Analyze all the allocated tasks are completed and discuss the success of the sprint. Perform a client demo.

Daily Standup

Daily Scrum Meeting

Time Box (15 min)

Same place

Same time

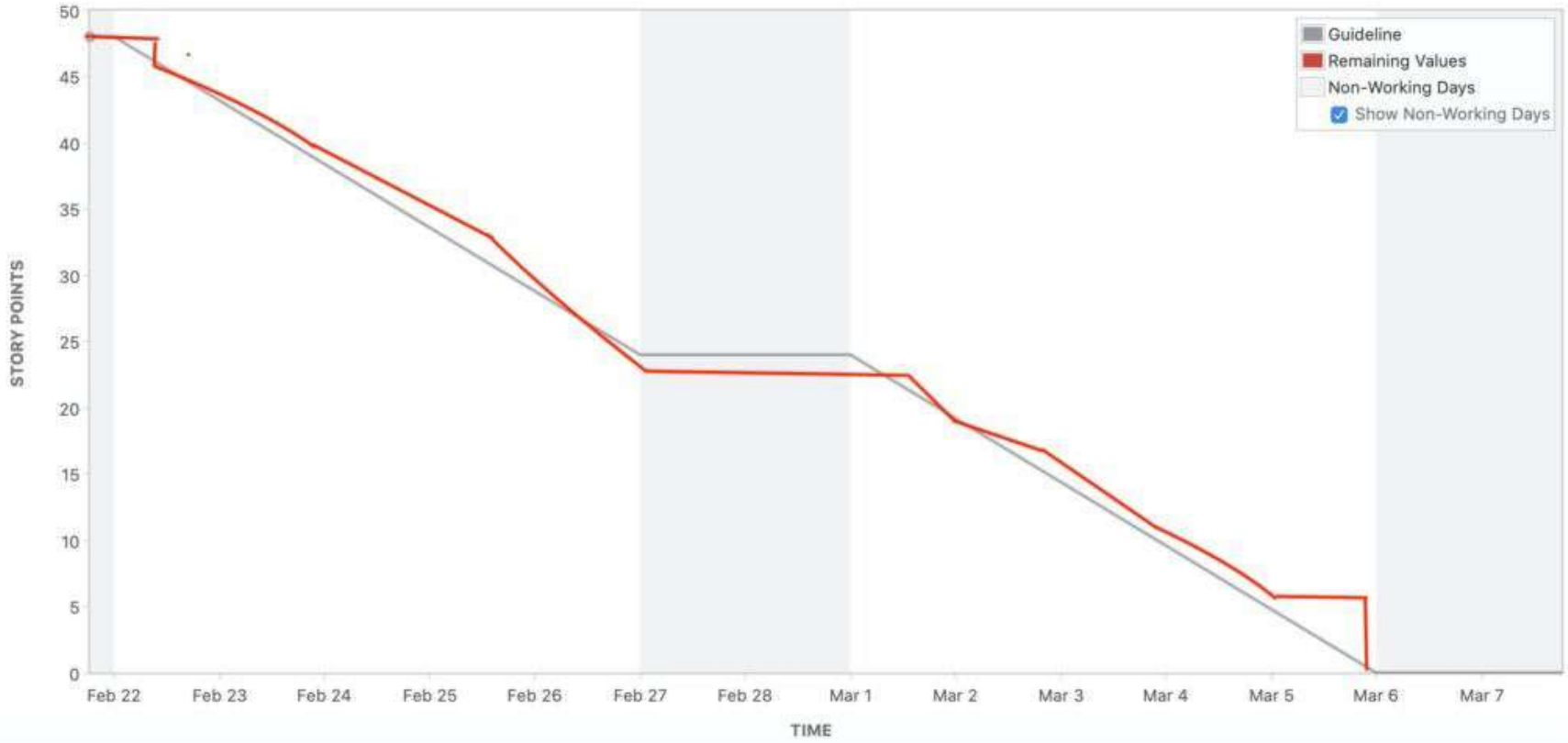
Facilitated
by Scrum Master

3 Focus upon 3 questions:

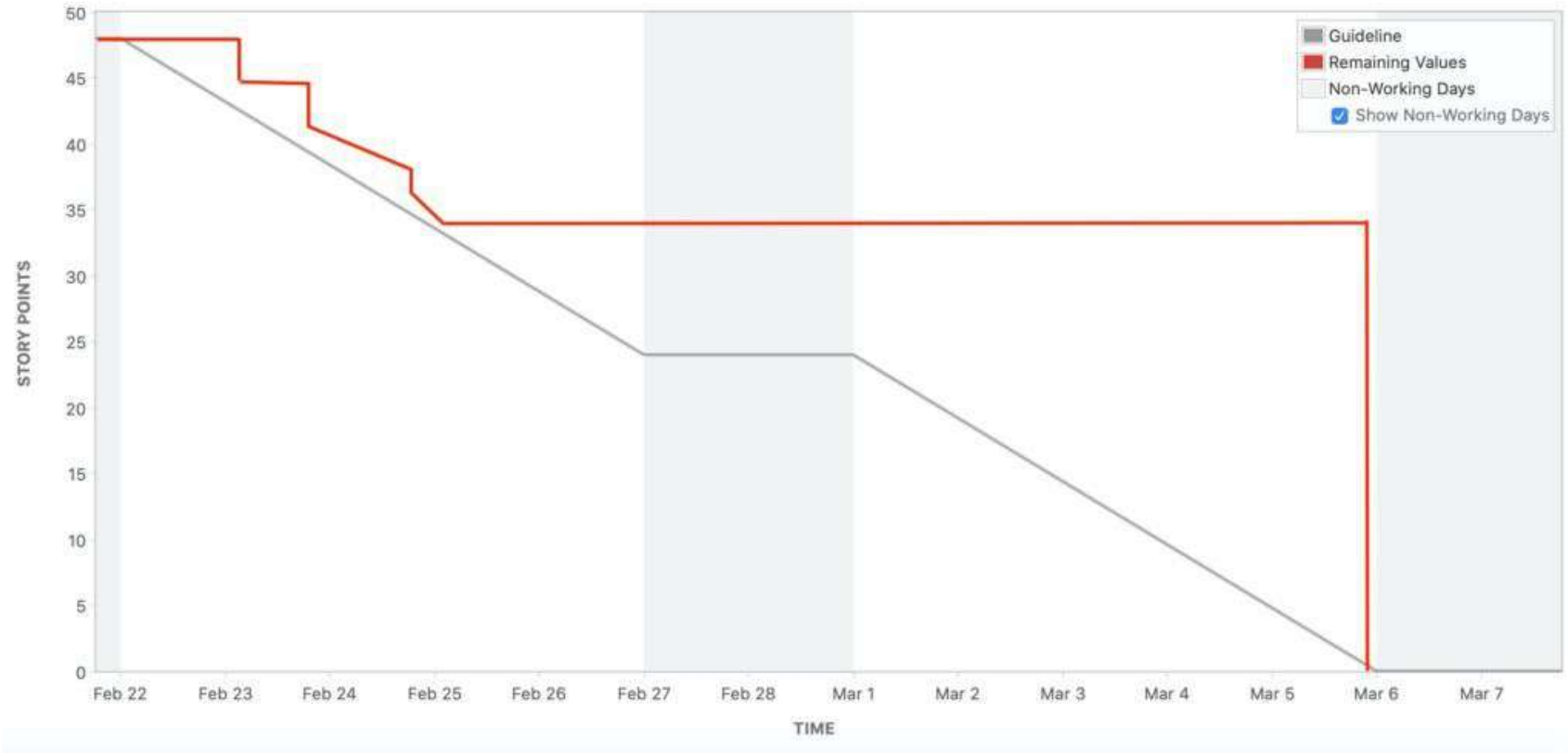
- What did I do yesterday?
- What will I do today?
- Is there any impediment?

Daily Standup

Burn Down Chart



Burn Down Chart



Agile Ceremonies and Process

- Sprint is around 2- 3 weeks

Backlog Grooming

PO and the Team analyze the backlog and prioritize the backlog

Daily Standup

Team have a quick daily standup to discuss the progress and blocks. Inform what was done previous day and what is planned for today.

Sprint Start

Sprint End

Week before sprint start

Daily

Sprint Planning

Assign the tasks to each team member and allocate relevant estimated times

Retrospective

Analyze all the allocated tasks are completed and discuss the success of the sprint. Perform a client demo.

Sprint Retrospective

Sprint Retrospective

Release: DOD Checklist
☐ Code completion
☐ Release environment made ready
☐ Unit test checked and green
☐ The functional test checked and green
☐ QA done, issue resolved if any
☐ To do annotations - resolved
☐ All teams members approve work
☐ No work left pending in any stage is checked
☐ Continuous integration and TDD verified

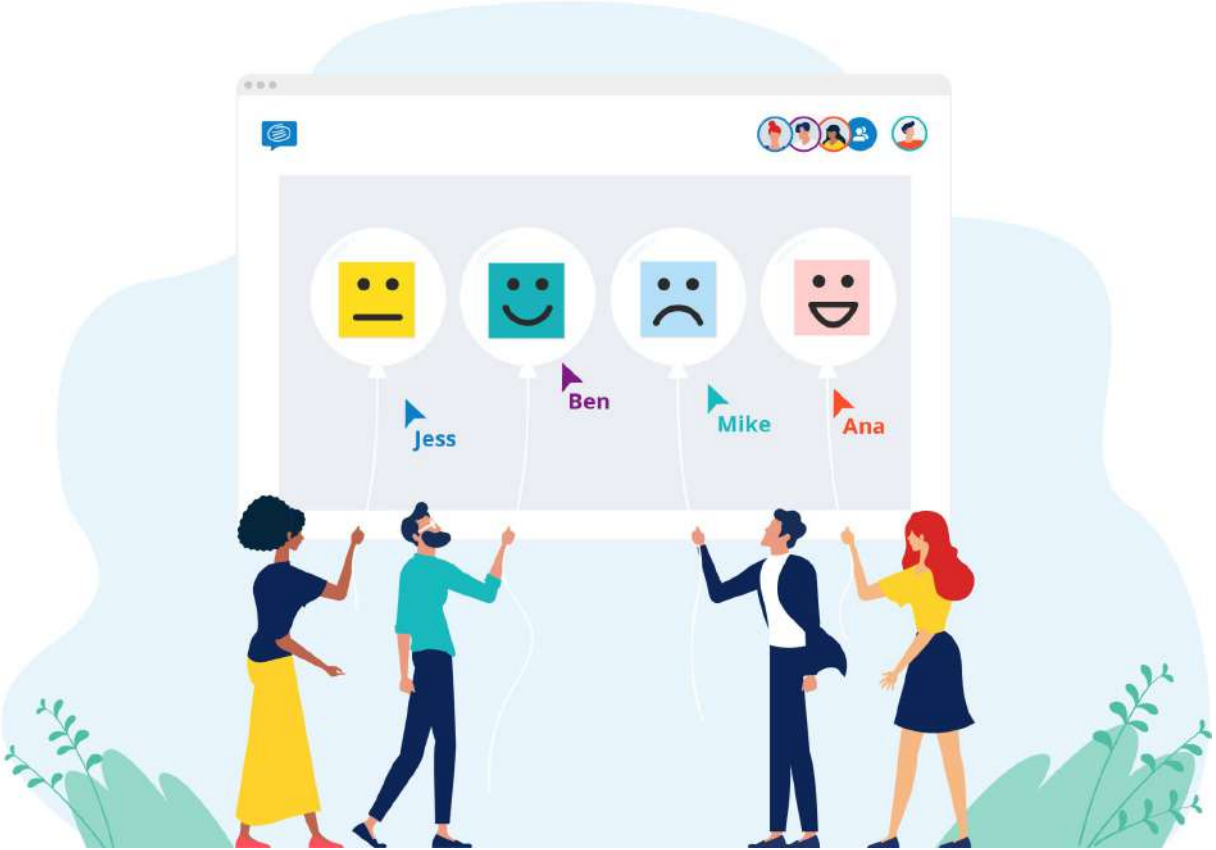
Spillover

Sprint Retrospective

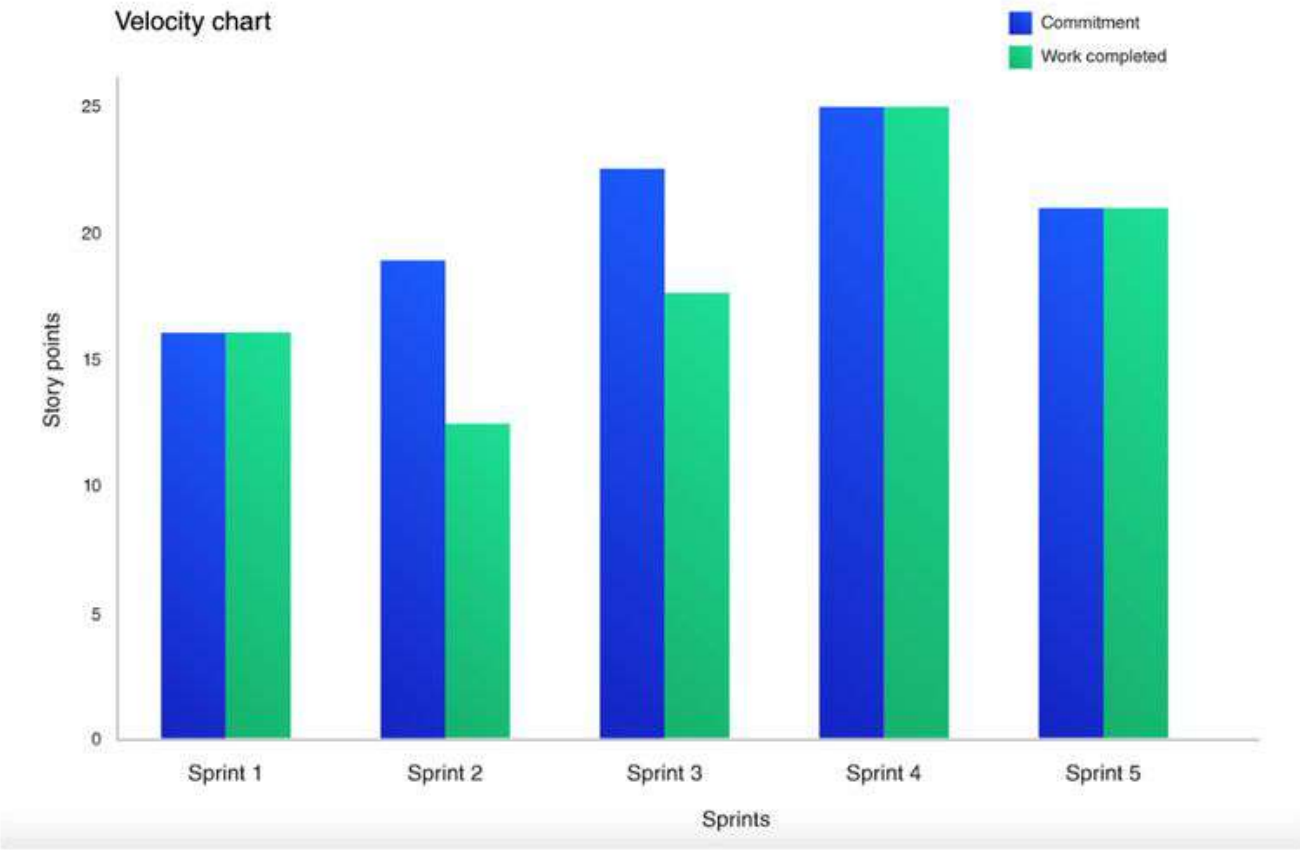
Sprint Retrospective Model

What worked well?	What didn't go well or as expected?	What should we try doing next? (learnings)
Example feedback: • Feedback from the marketing team was helpful • New smaller teams are more agile • First customers to test release loved it	Example feedback: • Unexpected bug after release • Some issues took longer to complete than expected	Example feedback: • Invite marketing to the sprint review • Test releases before launching on new domains

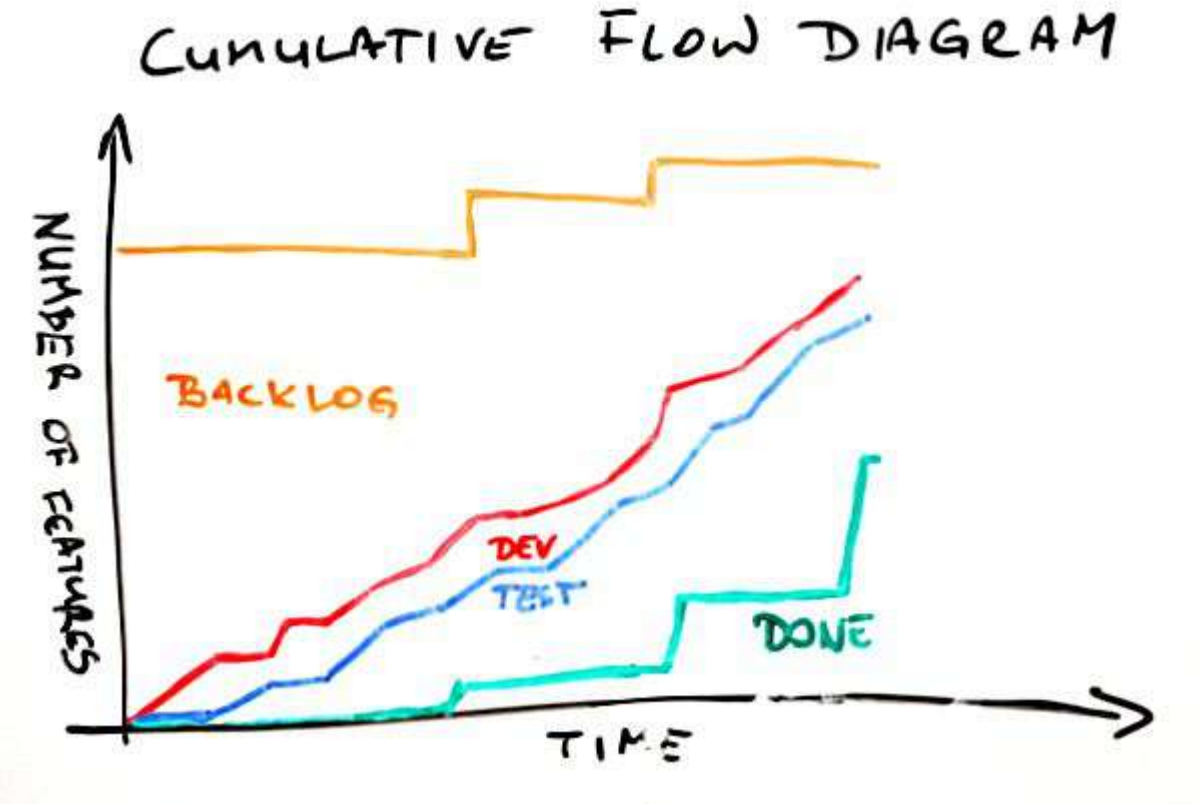
Sprint Retrospective



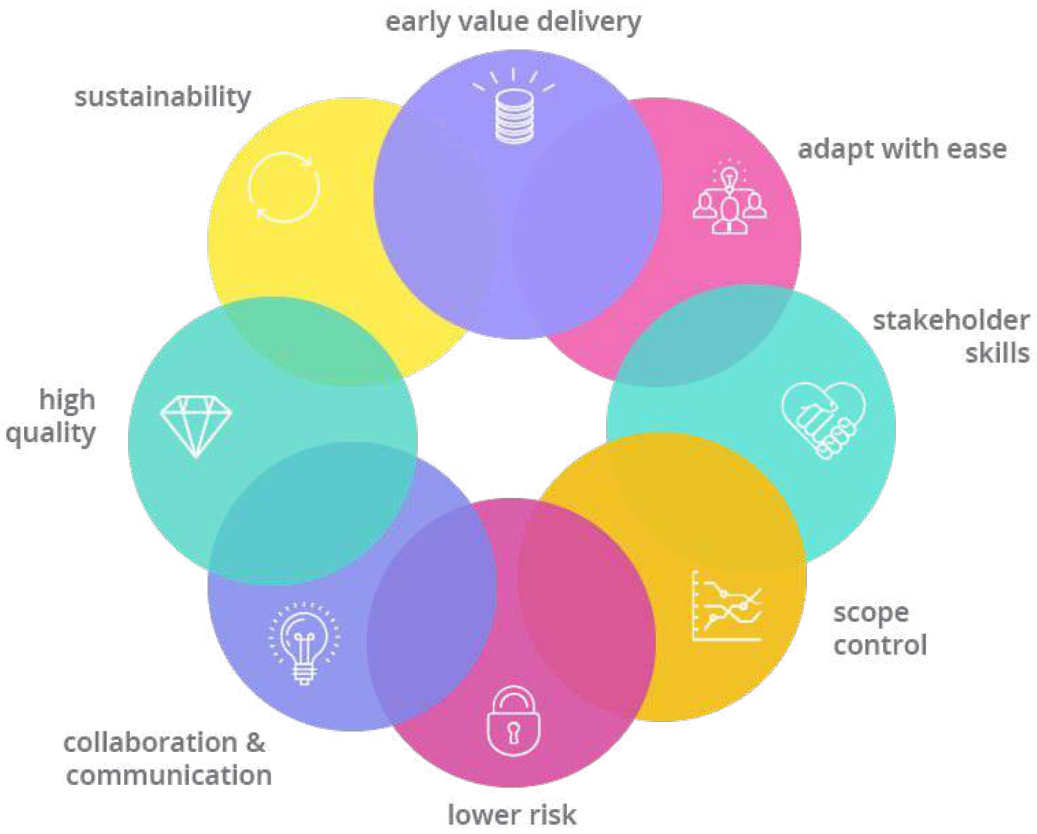
Team Velocity



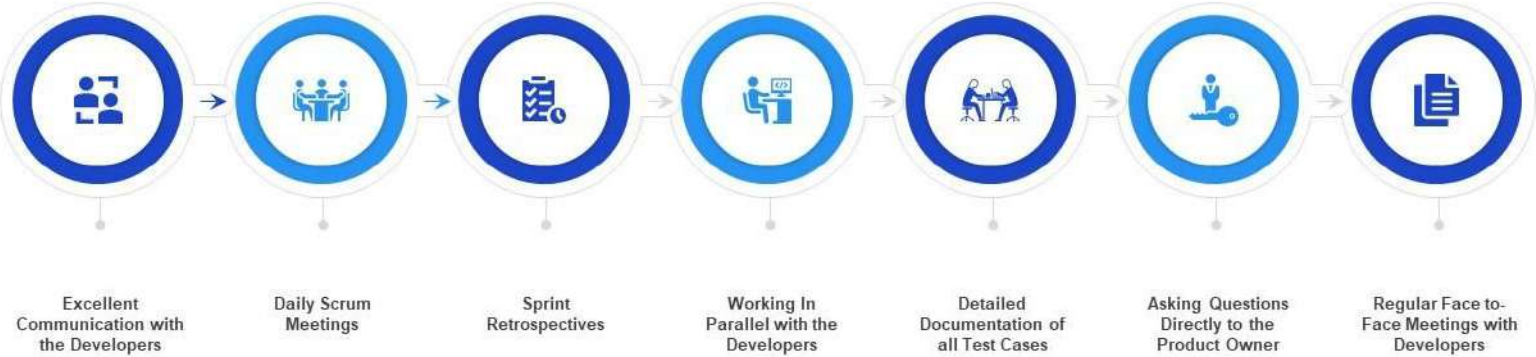
Cumulative Flow Diagram



Agile Advantages



QA Process in Agile



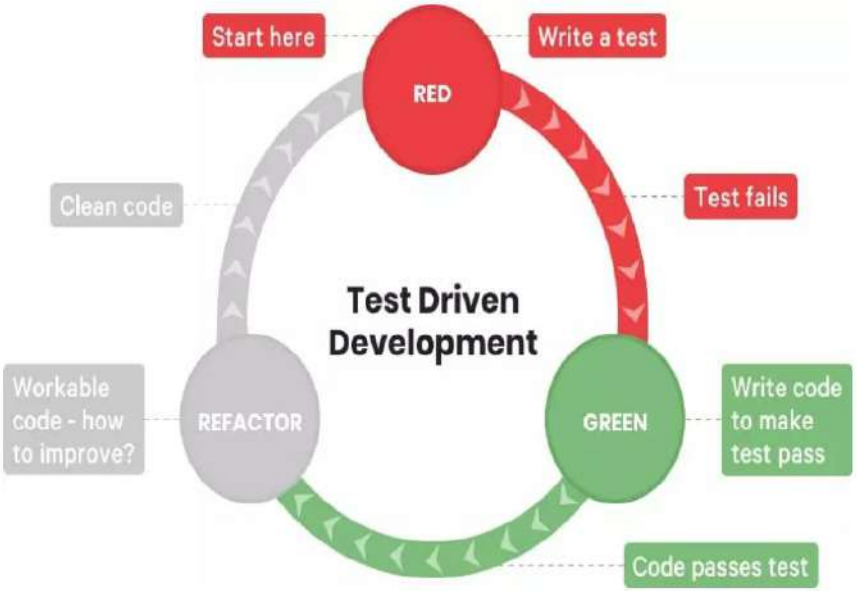
- No communication gap between QA and developers
- Daily meetings to monitor the growth of the product
- QA team will work along with the developers
- To be in touch with developers there should be face to face meetings with developers
- Add text here Add text here Add text here

Agile Advantages

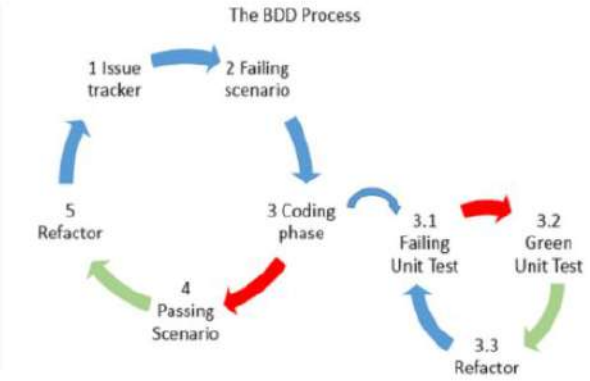
- The interaction between QA/BA & Dev is higher
- The time taken to identify and to resolve issues were reduced.
- Workload was spread out evenly throughout the release hence no one was exhausted.
- Functional piece of software provided to the client at the end of sprint

TDD and BDD in Agile

TDD Approach

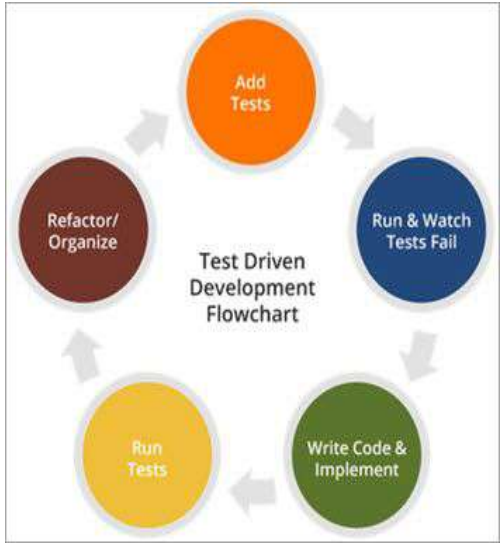


BDD Approach



What is TDD?

- Test Driven Development (TDD) is an ,
  - Iterative development process
  - Each iteration starts with a set of tests written for a new piece of functionality
  - These tests are suppose to fail during the start of iteration as there will be no application code corresponding to the tests.
  - Next phase of the iteration application code is written with an intention to pass all the tests written earlier in the iteration.
  - Once the application code is ready , tests run.
  - Any failures in the tests are marked and more application code is written /re-factored to make these tests pass.
  - Once application code are added / re-factored, the tests are run again.



TDD Example

```
public class LoginPage{

String username;
String password;

//store username
public void enterUserName(String username){
this.username = username;
}

//store password
public void enterPassword(String password){
this.password = password;
}

//match username and passowrd in db and return home page
public HomePage submit(){
if(username.existsInDB()){
String dbPassword = getPasswordFromDB(username);
if(dbPassword.equals(password){
Return new HomePage();
}
}
}
```

Benefits and Drawbacks of TDD

Benefits

Unit test proves that the code actually works

Refactoring allows to improve the design of the code

Low- level regression test suite

Test first reduce the costs of the bugs

Drawbacks

Developer can consider it is a waste of time

Test become part of the maintenance overhead of a project

Re-write the tests when requirement change

What is BDD?

- Behavior Driven Development (BDD) is a ,
  - Software Development Approach
  - Technique of Agile Software Development
  - Extension to TDD (Test Driven Development)
  - In BDD “acceptance tests are the starting point for SW design
  - Requirement should be converted in to user stories that can define tangible examples
  - Step definitions are generated using user stories with BDD language like Gerkin, Cucumber
  - Uses simple text language (English)
  - Bridge the gap between technical and non technical users

BDD is an Executable Specification

BDD Development process

Write the behavior of the application.

```
Scenario: A first scenario in which the feature can be exercised

    Given some precondition is met
      And another precondition
     When some action is performed
     Then the results should be as expected
      And some other testable outcome is achieved

Scenario: This is a second sample scenario

    ...
```

Scenario: Login check
Given I am on the login page
When I enter "username" username
And I enter "Password" password
And I click on the "Login" button
Then I am able to login successfully.

BDD Development process

Write the automated test script for this behavior

```
@RunWith(Cucumber.class)
public class MyStepDefinitions {

@Steps
LoginPage loginPage;

@Steps
HomePage hp;

    @Given("^I am on the login page $")
    public void i_am_on_the_login_page(){
        loginPage.gotoLoginPage();
    }

    @When("^I enter \"([^\"]*)\" username$")
    public void i_enter_something_username(String username) {
      loginPage.enterUserName(username);
    }

    @When("^I enter \"([^\"]*)\" password$")
    public void i_enter_something_password(String password) {
       loginPage.enterPassword(password);
    }

    @When("^I click on the \"([^\"]*)\" button$")
    public void i_click_on_the_submit_button(String strArg1) {
    hp =   loginPage.submit();
    }

    @Then("^I am able to login successfully\.$")
    public void i_am_able_to_login_successfully() {
      Assert.assertNotNull(hp);
    }
}
```

BDD Development process

Implement the functional code.

```
public class LoginPage{
String username = "";
String password = "";
//store username
public void enterUserName(String username){
this.username = username;
}
//store password
public void enterPassword(String password){
this.password = password;
}
//match username and passowrd in db and return home page
public HomePage submit(){
if(username.existsInDB()){
String dbPassword = getPasswordFromDB(username);
if(dbPassword.equals(password){
Return new HomePage();
}
}
}
```

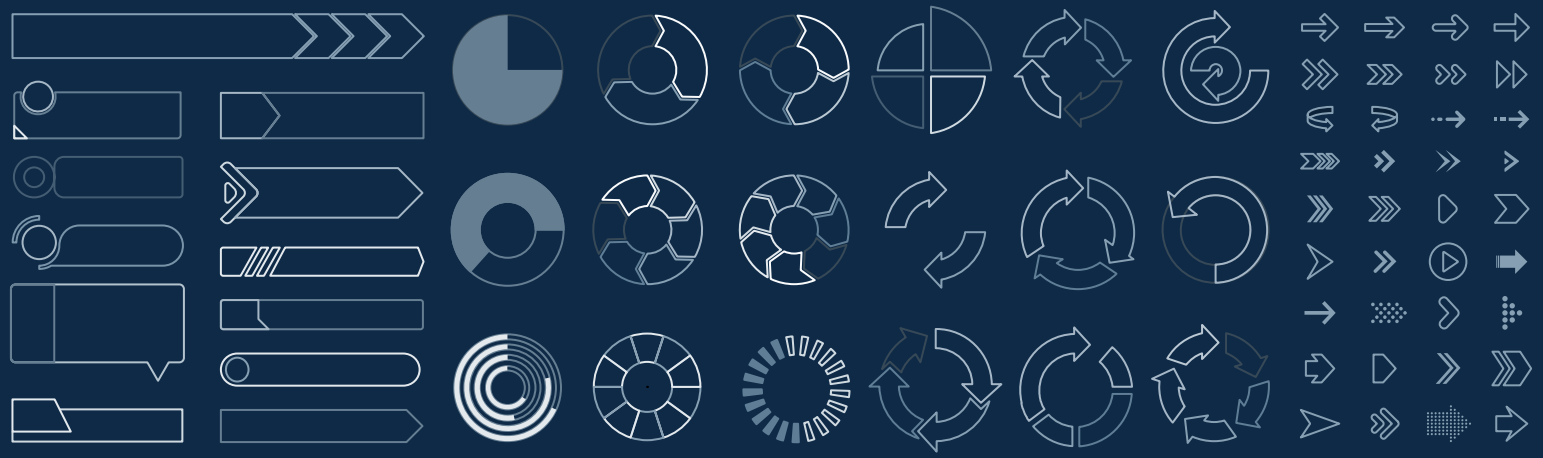
Benefits of BDD

- Strong Collaboration: Thanks to the shared language, product owners, developers, and testers all have in-depth visibility of the project's progression.
- Shorter Learning Curve: Since BDD is explained using very simple language, there's a much shorter learning curve for everyone involved.
- High Visibility: Being a nontechnical process by nature, behavior-driven development can reach a much wider audience.
- Rapid Iterations: BDD allows you to respond quickly to any and all feedback from your users so you can make improvements to meet their needs.
- BDD Test Suite: Similar to adopting TDD, adopting BDD gives your team confidence in the form of a test suite.
- Eliminate Waste: BDD allows you to clearly communicate requirements so there's less rework due to misinterpreted requirements and acceptance criteria.
- Focus on User Needs: Satisfied users are good for business and the BDD methodology allows user needs to be met through software development.
- Meet Business Objectives: With BDD, each development can be traced back to the actual business objectives.

Difference between TDD and BDD

TDD	BDD
1.Starts with writing a test case 2.Focus on how the functionality is implemented 3.Test cases written in programming language 4.Changes to application functionality impact test cases 5.Collaboration required only between developers 6.Better approach for projects which involve API and third -party tools 7.Tools support TDD:JUnit, TestNG,Nunit etc. 8.Require programming knowledge to understand the Test cases 9.Reduce the likelihood of having defects in test cases	1.Starts writing scenario as per the expected behavior 2.Focus on behavior of the application to end user 3.Written in Simple English – more readable 4.Scenarios not much impacted by changes to functionality 5.Collaboration required between all stakeholders 6.Better approach for projects driven by user actions eg: E-commerce applications 7.Tools support BDD : Cucumber, Mspec 8.Can be understood without programming knowledge 9.Defects in test cases are difficult to track compared to TDD

Agile Testing Case Study



Case Study

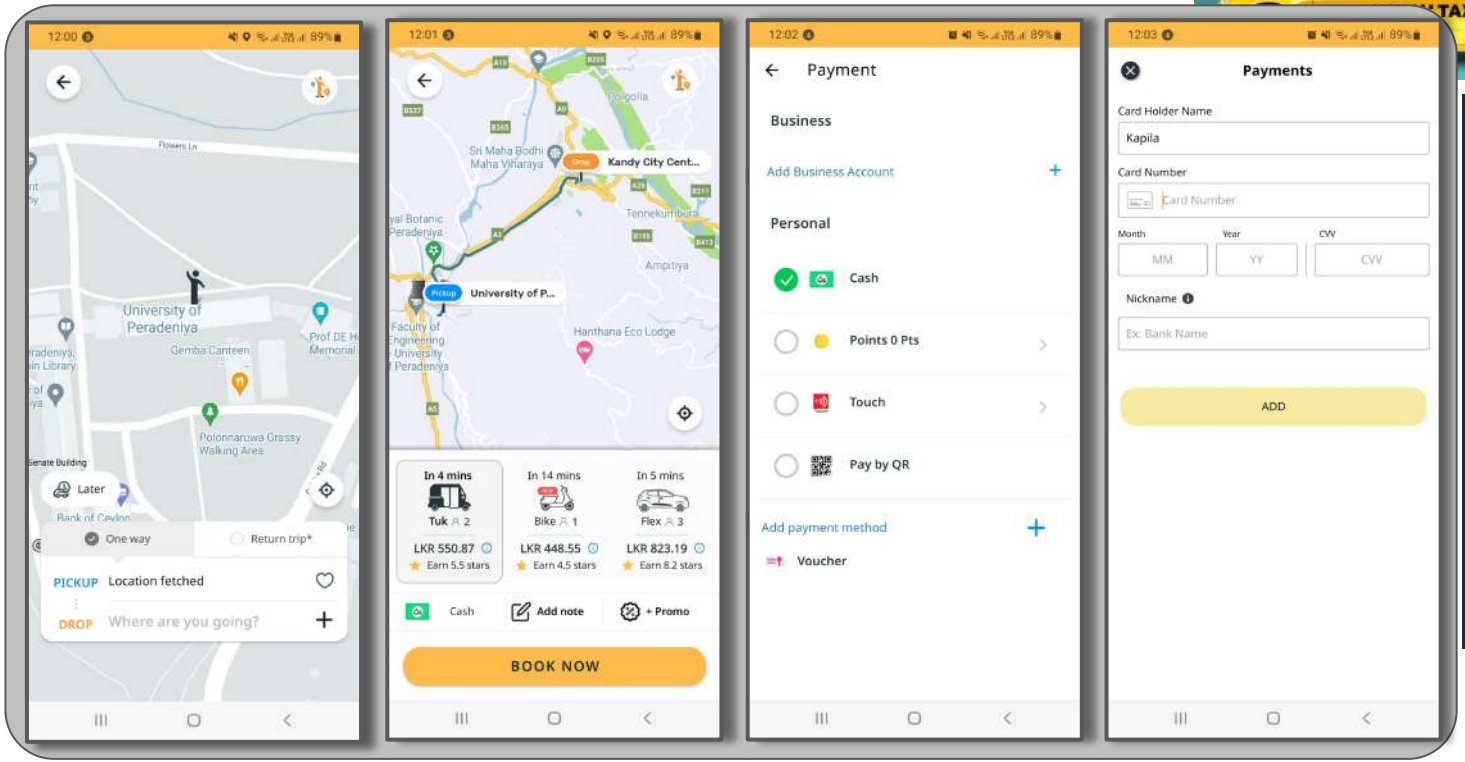
- Identify Challenges in Agile Development

Challenges in Agile Development

1. Resistance To Change
2. Lack Of Clarity
3. Reluctance To Empower Team Members
4. Robust Daily Communication
5. Taking The Team's Focus
6. The Comfort Of 'Going With The Flow'
7. Insufficient Support From Leaders
8. Lack Of Cross-Departmental Buy-In/Barriers
9. Estimating Upcoming Work
10. Fitting The Right Approaches /Misconceptions About Structure
11. The Buildup Of Silos
12. The Need of right Skilled 'Champions'
13. Conflict With Traditional Management Techniques
14. An Unwillingness To Collaborate
15. Slowdowns Caused By Existing Processes
16. Project Financing

- ✔ **Case Study**

A Taxi Booking Application



Simulate a agile team using Scrum framework for implementing below application

Agile Certifications

Questions ?

Quiz Time

Thanks !